UNIVERSIDAD NACIONAL

SEDE REGIONAL BRUNCA – CAMPUS PÉREZ ZELEDÓN

CURSO: PARADIGMAS DE PROGRAMACIÓN

PROYECTO FINAL

GGEZ: *Lenguaje de programación con vocabulario de videojuegos para estudiantes de primer ingreso*



Integrantes:

Steven Mata Quirós

Maikel Segura Quirós

Angel Ureña Naranjo

Profesor:

Pablo A. Venegas Elizondo

II Ciclo, 2026

Tabla de contenidos

1 Introducción **3**

2 Planteamiento del problema **4**

3 Evidencias de investigación **6**
3.1 Observación del contexto . . . 6
3.2 El vocabulario de los videojuegos como puente . . . 6
3.3 Evidencia del propio proyecto . . . 6

4 Propuesta de solución **8**
4.1 Usuarios . . . 8
4.2 Contexto de uso . . . 8
4.3 Alcance . . . 8
4.4 Limitaciones . . . 8
4.5 Beneficios esperados . . . 8
4.6 Características principales . . . 9
4.7 Ejemplos de programas que podrían desarrollarse . . . 9

5 Objetivos **12**
5.1 Objetivo general . . . 12
5.2 Objetivos específicos . . . 12

6 Alcance y limitaciones **13**
6.1 Alcance . . . 13
6.2 Limitaciones . . . 15

7 Descripción del lenguaje **16**
7.1 Identidad . . . 16
7.2 Palabras reservadas . . . 17
7.3 Sintaxis . . . 21
7.3.1 Instrucciones . . . 21
7.3.2 Condicionales . . . 22
7.3.3 Ciclos . . . 23
7.3.4 Funciones . . . 23
7.3.5 Operaciones aritméticas . . . 24
7.3.6 Operaciones lógicas . . . 24
7.3.7 Operaciones relacionales . . . 24
7.3.8 Clases, registros y errores . . . 24
7.4 Semántica . . . 25
7.5 Tipos de datos . . . 27
7.5.1 Tipos simples . . . 27
7.5.2 Tipos compuestos . . . 28

8 Funciones **29**
8.1 Categorías de funciones en GGEZ . . . 29
8.2 Buenas prácticas . . . 30
8.3 Restricciones del sistema de funciones . . . 30

9 Estructuras de control **31**
9.1 Cuadro sinóptico . . . 31
9.2 Recomendaciones de estilo . . . 31

9.3 Interrupción de ciclos 32

10 Operaciones **33**
10.1 Precedencia de operadores 33
10.2 Uso de llaves para agrupar 33
10.3 Evaluación de cortocircuito 33

11 Organización de archivos **34**
11.1 Estructura de un proyecto 34
11.2 Gestión desde el aplicativo 34
11.3 Validaciones del archivo principal 34
11.4 Convenciones de nomenclatura 35

12 Importaciones **36**
12.1 Forma de importación 36
12.2 Validaciones del compilador 36
12.3 Restricciones estructurales 37

13 Proceso de compilación **38**
13.1 Fases del compilador 38
13.2 Herramientas técnicas 38
13.3 Reporte de errores 39

14 Proceso de ejecución **40**
14.1 Estrategia general 40
14.2 Flujo de ejecución 40
14.3 Gestión de alcances 41
14.4 Ejecución de importaciones 41
14.5 Limitaciones de la ejecución 41

15 Manejo de errores **42**
15.1 Categorías de errores 42
15.2 Formato estándar de los mensajes 42
15.3 Filosofía de retroalimentación 43
15.4 Comportamiento del aplicativo 43
15.5 Advertencias 43

16 Innovación **45**
16.1 Descripción funcional 45
16.2 Componentes 45
16.3 Funcionamiento técnico 47
16.4 Validaciones e integración 48
16.5 Justificación 48

17 Arquitectura del aplicativo **49**
17.1 Tecnologías utilizadas 49
17.2 Organización en capas 49
17.3 Estructura de módulos 50
17.4 Flujo de una operación típica 50
17.5 Distribución 50
17.6 Consideraciones de diseño 51

18 Pruebas **52**

18.1 Programas válidos 52
18.2 Errores sintácticos 53
18.3 Errores semánticos 54
18.4 Pruebas de importaciones 55
18.5 Prueba de importación circular 56
18.6 Prueba completa relacionada con el problema 56
18.7 Advertencias 57

19 Contribuciones de los integrantes **59**

20 Conclusiones **60**

21 Referencias **61**

22 Anexos **62**
22.1 Manual de usuario de GGEZ 62

1. Introducción

El presente documento describe el diseño y la implementación de GGEZ, un lenguaje de programación desarrollado como proyecto final del curso Paradigmas de Programación de la Universidad Nacional, Sede Regional Brunca, durante el II Ciclo de 2026. GGEZ surge de una situación que el equipo conoce de cerca: la mayoría de las personas que ingresan a la carrera de Ingeniería en Sistemas juegan algún tipo de videojuego, y muchas de ellas entran pensando en programar sus propios juegos. Sin embargo, su primer contacto con la programación ocurre con lenguajes cuyo vocabulario y símbolos no se parecen a nada de lo que ya conocen.

A diferencia de los lenguajes tradicionales, GGEZ toma su vocabulario de la jerga de los videojuegos. Su nombre viene de la expresión *GG EZ* («good game, easy»), que los jugadores usan al terminar una partida; sus bloques se abren con `glhf` («good luck, have fun») y se cierran con `gg`; las variables se guardan con `stash`, las funciones son `combo`, un ciclo se abandona con `ragequit` y se pasa a la siguiente vuelta con `respawn`. El entorno de desarrollo sigue la misma idea: programar se vuelve una partida con maná, niveles, estrellas, logros y un mundo que avanza a medida que el código del estudiante funciona.

A lo largo de este documento se detalla, en primer lugar, el problema identificado y las evidencias que respaldan su relevancia. Posteriormente se expone la propuesta de solución, los objetivos del proyecto y su alcance. La sección central del documento describe formalmente el lenguaje: su identidad, palabras reservadas, sintaxis, semántica y tipos de datos. Finalmente se documentan los procesos de compilación y ejecución, el manejo de errores, la característica de innovación, la arquitectura del entorno de desarrollo integrado, las pruebas realizadas y las contribuciones individuales de cada integrante del equipo.

El proyecto es desarrollado por Steven Mata Quirós, Maikel Segura Quirós y Angel Ureña Naranjo, bajo la orientación del profesor Pablo A. Venegas Elizondo.

2. Planteamiento del problema

Aprender a programar exige, al mismo tiempo, comprender conceptos abstractos (variables, ciclos, funciones, objetos) y memorizar las reglas de un lenguaje formal lleno de símbolos (`{ }`, `;`, `&&`, `!=`, `++`). Para quien recién ingresa a la universidad, ese doble esfuerzo coincide con el momento de mayor incertidumbre de la carrera: todo es nuevo, el ritmo es distinto al del colegio y el primer curso de programación suele ser el que define si la persona continúa o no.

El problema

El problema que aborda este proyecto es la **distancia entre el mundo que el estudiante de primer ingreso ya domina, los videojuegos, y la forma en que se le presenta la programación**. El estudiante llega sabiendo qué es un inventario, un combo, un *respawn* o abandonar una partida, y muchas veces eligió la carrera porque quiere crear juegos. Sin embargo, el primer curso le presenta un lenguaje que habla otro idioma y un entorno de desarrollo que no ofrece ninguna sensación de avance más allá de que el programa compile o no. Esa distancia se traduce en desmotivación, sensación de no progresar y, en algunos casos, abandono temprano.

A quiénes afecta

Afecta principalmente a los **estudiantes de primer ingreso de Ingeniería en Sistemas** y carreras afines que llevan su primer curso de programación. Dentro de ese grupo afecta en especial a quienes no programaron en el colegio y a quienes entraron a la carrera motivados por los videojuegos y se encuentran con un primer curso que no se parece a lo que esperaban. De forma secundaria afecta a los docentes de cursos introductorios, que necesitan mantener la motivación de grupos numerosos y con niveles muy distintos.

En qué contexto ocurre

El problema se manifiesta en dos contextos. El primero es el aula universitaria del primer ciclo, donde el lenguaje lo define el curso y el estudiante debe aprender, a la vez, la lógica de programación y la sintaxis de un lenguaje profesional. El segundo es la práctica fuera de clase: el estudiante resuelve ejercicios solo, recibe mensajes de error difíciles de entender y no tiene una señal clara de que está mejorando, algo que contrasta con la retroalimentación inmediata y constante a la que está acostumbrado en los juegos.

Qué soluciones existen actualmente

Existen varias herramientas conocidas que buscan acercar la programación a principiantes:

- **Lenguajes visuales por bloques**, como Scratch, donde el programa se arma uniendo piezas gráficas en lugar de escribir código.
- **Juegos que enseñan a programar**, como CodeCombat, donde el jugador controla un personaje escribiendo código en un lenguaje profesional.

- **Lenguajes profesionales usados en el curso**, como Java, C# o Python, acompañados de entornos de desarrollo de uso general.

Qué limitaciones presentan

- Los lenguajes por bloques están pensados principalmente para niños y adolescentes; un estudiante universitario puede percibirlos como poco serios, y el paso de los bloques a un lenguaje escrito no es directo.
- Los juegos que enseñan a programar usan el lenguaje profesional sin cambios: el juego es el envoltorio, pero el vocabulario sigue siendo ajeno al estudiante. Además, solo se resuelven los retos que el juego trae, no programas propios.
- Los lenguajes profesionales y sus entornos están hechos para programadores con experiencia: sus mensajes de error asumen conocimientos que el principiante todavía no tiene y no ofrecen ningún elemento de motivación.
- Ninguna de estas opciones usa el vocabulario de los videojuegos dentro del propio lenguaje, ni conecta un sistema de progreso con cualquier programa que el estudiante escriba.

Cómo GGEZ contribuirá a atender el problema

GGEZ se propone como un lenguaje textual completo, con la misma estructura que un lenguaje imperativo y orientado a objetos como Java, pero cuyo vocabulario sale de los videojuegos. Así, cada concepto nuevo se asocia con algo que el estudiante ya conoce: `respawn` vuelve a empezar la vuelta del ciclo, `ragequit` lo abandona, `clutch` y `revive` rescatan un error y `spawn` crea un objeto. Como la estructura de los programas es la de un lenguaje convencional, lo aprendido se transfiere a Java, C# o Python, y la ayuda del aplicativo muestra cada ejemplo de GGEZ junto a su equivalente en Java.

El entorno de desarrollo completa la propuesta: cada ejecución gasta maná, avanza misiones en un mundo de diez zonas, desbloquea logros y temas visuales, y los niveles con pruebas automáticas dan estrellas. Ese progreso solo avanza cuando el código del estudiante corre de verdad, de modo que la motivación del juego queda unida al aprendizaje real. De esta manera, GGEZ busca ser un puente entre las ganas de programar juegos con las que llega el estudiante y los fundamentos de programación que necesita aprender.

3. Evidencias de investigación

Esta sección presenta las evidencias que fundamentan las decisiones de diseño de GGEZ. El equipo no realizó un estudio formal con encuestas ni cuenta con fuentes bibliográficas propias; las evidencias provienen de la observación directa del contexto, del análisis del vocabulario de los videojuegos y de los resultados verificables del propio proyecto.

3.1 Observación del contexto

Los tres integrantes del equipo son estudiantes de la carrera y vivieron el primer curso de programación. Desde esa experiencia se observó que:

- La mayoría de los compañeros de primer ingreso juega videojuegos con regularidad y conoce su vocabulario en inglés (*gg*, *afk*, *nerf*, *buff*, *respawn*), aunque no hable inglés con fluidez.
- Una motivación frecuente para entrar a la carrera es querer crear videojuegos.
- Las primeras dificultades suelen ser de forma y no de lógica: llaves sin cerrar, puntos y comas olvidados, operadores confundidos (= con ==) y mensajes de error que no se entienden.
- La sensación de no avanzar desmotiva más que la dificultad en sí: un ejercicio que no compila no deja ninguna señal de progreso.

3.2 El vocabulario de los videojuegos como puente

Se analizó la jerga de los videojuegos buscando, para cada construcción de un lenguaje convencional, una palabra que el jugador ya asocie con un significado parecido. El resultado muestra que la correspondencia es natural en la mayoría de los casos: un `combo` es una secuencia de acciones con nombre, como una función; `respawn` es volver a empezar, como *continue*; `ragequit` es abandonar, como *break*; `afk` es estar ausente, como *null*; `buff` y `nerf` son mejorar y empeorar, como `++` y `--`; y un `guild` agrupa miembros, como una clase. La lista completa de palabras reservadas, con su propósito y un ejemplo, está en la sección 7.2.

Además, al usar palabras para los operadores relacionales (`over`, `under`, `matches`), los símbolos `<` y `>` quedan libres para índices y listas, y se elimina la confusión entre `=` y `==`: asignar es `set` y comparar es `matches`.

3.3 Evidencia del propio proyecto

La propuesta se verificó con resultados del proyecto que cualquiera puede reproducir:

- El lenguaje permite escribir todos los programas que se esperan en un primer curso (tipos, condicionales, ciclos, funciones, recursión, listas, registros, clases y proyectos de varios archivos), como muestran las pruebas de la sección 18.

- Cada error del compilador indica su posible causa en español, y el entorno lleva a la línea del error con un clic.
- El sistema de progreso del entorno solo avanza con ejecuciones reales, porque el intérprete informa qué construcciones se ejecutaron.

En síntesis, las evidencias respaldan las decisiones fundamentales del diseño de GGEZ: mantener un lenguaje escrito y con la estructura de los lenguajes profesionales, reemplazar los símbolos sin significado por palabras que el estudiante ya conoce, entregar mensajes de error con su posible causa y usar elementos de juego ligados a la ejecución real del código. El efecto de GGEZ sobre la motivación no se ha medido todavía con estudiantes reales; esa evaluación queda como trabajo futuro.

4. Propuesta de solución

GGEZ es un lenguaje de programación imperativo, estructurado y con soporte para programación orientada a objetos, de tipado estático y con vocabulario de videojuegos, acompañado de un entorno de desarrollo integrado (IDE) propio con mecánicas de juego. Esta propuesta detalla los usuarios a los que se dirige el lenguaje, el contexto en que se utilizará, los beneficios esperados, sus características principales y algunos ejemplos representativos de los programas que podrán desarrollarse en él.

4.1 Usuarios

El público objetivo principal de GGEZ son los **estudiantes de primer ingreso** de Ingeniería en Sistemas y carreras afines que llevan su primer curso de programación, en especial quienes juegan videojuegos o quieren programarlos. Como público secundario, GGEZ puede ser útil para docentes de cursos introductorios que buscan material de práctica con un componente de motivación, y para cualquier persona que quiera aprender a programar por su cuenta.

4.2 Contexto de uso

GGEZ está pensado para dos contextos. En el aula, como herramienta de práctica durante las primeras semanas del curso introductorio, antes o junto al lenguaje profesional del curso. Fuera del aula, para que el estudiante practique por su cuenta escribiendo programas pequeños y medianos (calculadoras, clasificadores, juegos por turnos, inventarios), en un IDE que funciona sin conexión a internet, en Linux y Windows, sin instalar nada aparte.

4.3 Alcance

El alcance del proyecto, en su versión de entrega, comprende el diseño e implementación completa de un lenguaje de programación con su compilador, su intérprete y su entorno de desarrollo integrado, cumpliendo los requerimientos establecidos en el enunciado del proyecto. La descripción detallada del alcance se encuentra en la sección 6.1.

4.4 Limitaciones

Esta primera versión de GGEZ se enfoca en cubrir con solidez los elementos de un lenguaje para un primer curso, priorizando un lenguaje coherente y completo por sobre una biblioteca extensa. Las limitaciones específicas se documentan en la sección 6.2.

4.5 Beneficios esperados

- **Menor fricción inicial.** Las palabras reservadas evocan algo que el estudiante ya conoce, y la ayuda muestra cada construcción junto a su equivalente en Java.

- **Motivación sostenida.** El progreso del mundo, los logros y las estrellas dan una señal visible de avance cada vez que el código funciona.
- **Retroalimentación comprensible ante errores.** Cada mensaje indica tipo de error, archivo, línea, descripción y posible causa, y un clic lleva a la línea.
- **Transición facilitada a lenguajes profesionales.** Tipos explícitos, funciones con firma, clases con visibilidad y herencia, y módulos: todo con la misma estructura que Java.
- **Buenas prácticas tempranas.** Proyectos con varios archivos, constantes, manejo de errores y advertencias de código sin usar.

4.6 Características principales

- **Vocabulario de videojuegos.** Las palabras reservadas están en inglés, como en la cultura de los juegos; los mensajes de error, la ayuda y la documentación están en español.
- **Bloques delimitados por palabras.** Todo bloque abre con `glhf` y cierra con `gg`; la indentación no es obligatoria.
- **Tipado estático.** Toda variable, parámetro y retorno declara su tipo, y los tipos se verifican antes de ejecutar.
- **Lenguaje completo para un primer curso.** Cinco tipos simples, el valor nulo `afk`, vectores (`squad`), registros (`inventory`), clases con herencia (`guild`), funciones con recursión, selección múltiple, tres tipos de ciclo y manejo de errores.
- **Proyectos de varios archivos.** Importación con `travel` y detección de importaciones circulares.
- **Mensajes de error con posible causa**, más advertencias de variables y funciones sin usar y de código que nunca se ejecuta.
- **Entorno de desarrollo integrado propio.** Árbol de archivos, editor con pestañas, minimapa, autocompletado, coloreado de sintaxis, búsqueda, terminal integrada, panel de problemas, ayuda del lenguaje y botones de compilar (`F6`) y ejecutar (`F5`).
- **Innovación.** Gamificación ligada a la ejecución, terminal real con asistente de inteligencia artificial y sistema de extensiones, documentados en la sección 16.

4.7 Ejemplos de programas que podrían desarrollarse

Los siguientes cuatro ejemplos ilustran el tipo de programas que un usuario puede desarrollar en GGEZ. Cubren instrucciones básicas, control de flujo, funciones, listas e importación entre archivos.

Ejemplo 1: Cálculo del daño de un ataque

Programa simple que calcula el daño de un golpe crítico.

```
lobby.launch{start THE GAME} glhf
    stash score ataque set 25
    stash aim critico set 1.5
    chat.send{"Dano: " gain {ataque crit critico}}
gg
```

Ejemplo 2: Rango de un jugador

Programa que clasifica una puntuación según su rango utilizando un condicional múltiple dentro de una función.

```
combo rango{score puntos} drops lore glhf
    scan puntos atleast 90 glhf
        reward "S"
    gg fallback scan puntos atleast 70 glhf
        reward "A"
    gg fallback glhf
        reward "B"
    gg
gg

lobby.launch{start THE GAME} glhf
    chat.send{"Rango: " gain rango{85}}
gg
```

Ejemplo 3: Mochila con una lista

Programa que usa un `squad` para guardar objetos y recorrerlos con un ciclo.

```
lobby.launch{start THE GAME} glhf
    stash squad[lore] mochila set <"Espada" | "Escudo">
    mochila.recruit{"Pocion"}
    stash score i set 0
    farm i set 0 | i under mochila.size | i buff glhf
        chat.send{i gain ": " gain mochila<i>}
    gg
gg
```

Ejemplo 4: Modularización con importación entre archivos

Programa distribuido en dos archivos que demuestra el sistema de importación de GGEZ.

Archivo `lib/combate.ggez`:

```
combo danoTotal{score base | score golpes} drops score glhf
    reward base crit golpes
gg
```

Archivo `principal.ggez`:

```
travel combate from lib

lobby.launch{start THE GAME} glhf
    chat.send{"Dano total: " gain combate.danoTotal{20 | 3}}
gg
```

5. Objetivos

5.1 Objetivo general

Diseñar e implementar GGEZ, un lenguaje de programación con vocabulario de videojuegos y su entorno de desarrollo integrado con mecánicas de juego, orientado a facilitar y motivar el aprendizaje de la programación en estudiantes de primer ingreso de Ingeniería en Sistemas.

5.2 Objetivos específicos

Para alcanzar el objetivo general, el proyecto se propone los siguientes objetivos específicos:

- **Definir formalmente el lenguaje GGEZ**, especificando su identidad, palabras reservadas, gramática, semántica y sistema de tipos, tomando su vocabulario de los videojuegos.
- **Implementar un compilador funcional**, capaz de realizar análisis léxico, sintáctico y semántico, verificación estática de tipos y validación completa de las importaciones entre archivos, incluida la detección de importaciones circulares.
- **Desarrollar un intérprete** que ejecute los programas a partir del árbol sintáctico de una compilación exitosa, gestionando el flujo de control, las variables con sus alcances, las llamadas a funciones, los objetos, las importaciones y los errores.
- **Construir un entorno de desarrollo integrado propio**, con editor de código, árbol de archivos, segmento de salida con clasificación de mensajes, menús de consulta del lenguaje y botones dedicados para compilar y ejecutar.
- **Incorporar una característica de innovación** que refuerce el propósito del lenguaje, esté integrada al aplicativo y sea demostrable durante la presentación.
- **Documentar el proyecto en su totalidad**, incluyendo el diseño del lenguaje, la arquitectura del aplicativo, un conjunto representativo de pruebas y las contribuciones de cada integrante.

6. Alcance y limitaciones

Esta sección presenta, de forma sintética, las capacidades que el proyecto GGEZ ofrece en su versión actual y las que quedan fuera de su alcance. Funciona como referencia para verificar rápidamente el cumplimiento de los requisitos.

6.1 Alcance

El proyecto GGEZ, en su versión de entrega, comprende los siguientes componentes y capacidades:

Lenguaje de programación

- 71 palabras reservadas tomadas del vocabulario de los videojuegos
- Cinco tipos de datos simples (`score`, `aim`, `lore`, `pixel` y `win`) y el valor nulo `afk`
- Dos tipos de datos compuestos: vector (`squad`) y registro (`inventory`)
- Declaración de variables con tipo explícito, constantes y asignación con operadores compuestos
- Entrada y salida de datos
- Operaciones aritméticas (incluido el menos unario), relacionales y lógicas con cortocircuito
- Condicionales simple, con alternativa y múltiple, y selección múltiple (`loadout`)
- Ciclos equivalentes a *for*, *while* y *do-while*, con interrupción y salto de vuelta
- Funciones con y sin retorno, con parámetros tipados, y recursión
- Manejo de errores desde el código del usuario
- Clases con constructor, visibilidad, herencia simple y polimorfismo
- Punto de entrada explícito: `lobby.launch{start THE GAME}`
- Importación de funciones y tipos entre archivos del mismo proyecto

Compilador

- Análisis léxico, sintáctico, de importaciones y semántico
- Verificación estática de tipos antes de ejecutar
- Validación del archivo principal y de su punto de entrada único
- Validación completa de las importaciones, incluidas las circulares
- Reporte de errores (tipo, archivo, línea, columna, descripción y posible causa) y de advertencias

Intérprete

- Ejecución desde el punto de entrada recorriendo el árbol sintáctico
- Gestión de variables con alcance de bloque
- Llamadas a funciones y métodos, retorno de valores y objetos
- Ejecución de lo importado desde otros archivos
- Errores atrapables y límites de pasos (maná) y de recursión

Entorno de desarrollo integrado

- Interfaz gráfica con menú, árbol de archivos, editor y segmento de salida
- Identificación visible del archivo activo (pestaña y ruta) y del archivo principal (etiqueta «principal»)
- Gestión de proyectos: creación, apertura, creación de archivos y carpetas, renombrado y eliminación
- Menú de consulta del lenguaje: diccionario de palabras, tipos, operadores, estructuras de control, funciones, listas, errores, objetos, importaciones, semántica y ejemplos
- Editor con numeración de líneas, coloreado de sintaxis, autocompletado, minimapa, búsqueda y reemplazo, ir a línea, zoom y pestañas
- Compilación y ejecución desde botones dedicados, terminal integrada con entrada interactiva y pestañas de errores y problemas

Innovación

- Gamificación ligada a la ejecución real: maná, mundo con misiones, niveles con estrellas, monedas, pistas, logros y temas desbloqueables
- Terminal real con asistente de inteligencia artificial y sistema de extensiones (sección 16)

Gestión y pruebas

- Repositorio en GitHub con contribuciones de los tres integrantes, README, código, documentación y pruebas versionadas
- Batería de pruebas: 5 programas válidos, 5 con errores sintácticos, 5 con errores semánticos, 5 de importaciones, 1 de importación circular, 1 completa relacionada con el problema y 1 de advertencias
- Ejecutable único del aplicativo para Linux y Windows, generado con el script `publicar.sh`

6.2 Limitaciones

Las siguientes capacidades quedan fuera del alcance de esta versión del proyecto:

Del lenguaje

- Interfaces (`questline`), terminación explícita del programa (`quit`), organización en paquetes (`server`, `region`), pilas (`stack`) y colas (`queue`): son palabras reservadas para una versión futura y el compilador responde con un mensaje claro si se usan
- Biblioteca estándar `arsenal` (`physics`, `rng`, `clock`, `save`): reservada; en esta versión solo existe el módulo `chat`
- Conversión de texto a número: `chat.load` siempre devuelve un `lore`
- Inicialización de vectores de vectores con un literal anidado
- Genéricos definidos por el usuario, sobrecarga de funciones, funciones anónimas y concurrencia
- Manejo de archivos externos, comunicación en red y acceso a bases de datos

Del entorno de desarrollo

- Depurador con puntos de interrupción y ejecución paso a paso; el botón de pausa del panel de juego todavía no suspende el programa
- Formateo automático del código: la extensión «GGEZ Formatter» aparece en el catálogo, pero no está implementada
- Integración con sistemas de control de versiones

Del ámbito de aplicación

- Generación de ejecutables propios a partir de programas GGEZ: el lenguaje es interpretado
- Uso en entornos de producción de software
- El ejecutable de Windows se genera con el mismo script que el de Linux, pero no se ha probado en un equipo con Windows

7. Descripción del lenguaje

7.1 Identidad

Nombre

GGEZ.

Propósito

Ofrecer un lenguaje de programación completo cuyo vocabulario esté tomado de los videojuegos, para que el estudiante de primer ingreso asocie cada concepto de programación con algo que ya conoce, y acompañarlo de un entorno de desarrollo que convierta la práctica en una partida.

Usuarios

Estudiantes de primer ingreso de Ingeniería en Sistemas y carreras afines, en especial quienes juegan videojuegos o quieren programarlos; docentes de cursos introductorios que buscan material de práctica con un componente de motivación.

Extensión de los archivos

Los archivos de código fuente escritos en GGEZ utilizan la extensión `.ggez`. La elección responde a dos criterios: identidad (es el mismo nombre del lenguaje, por lo que no hay duda de qué contiene el archivo) y unicidad (no colisiona con extensiones de otros lenguajes o formatos). Un proyecto GGEZ se compone de uno o más archivos con esta extensión, uno de los cuales se designa como archivo principal.

Logotipo

El logotipo de GGEZ es el mismo ícono del entorno de desarrollo: una consola de videojuegos portátil en estilo *pixel art* con las letras «GGEZ», que resume en una imagen la idea del proyecto: programar como quien juega.



Filosofía general de diseño

GGEZ se rige por cuatro principios de diseño: (1) **vocabulario conocido**, usando palabras de la jerga de los videojuegos en lugar de símbolos sin significado para el estudiante; (2) **estructura profesional**, con tipos explícitos, funciones con firma, clases y módulos como en Java, para que lo aprendido se transfiera; (3) **retroalimentación clara**, con errores que indican dónde ocurrieron y su posible causa; y (4) **progreso visible**, con un entorno que premia cada programa que funciona.

7.2 Palabras reservadas

GGEZ cuenta con 71 palabras reservadas que constituyen su vocabulario. Se escriben en minúscula (salvo `THE` y `GAME` del punto de entrada), distinguen mayúsculas de minúsculas y no pueden utilizarse como nombres de variables, funciones, parámetros, clases ni módulos. La siguiente tabla presenta cada palabra reservada con su propósito, forma general de uso y un ejemplo.

Palabra reservada	Propósito	Forma de uso	Ejemplo
Estructura del programa			
`lobby` `launch`	Punto de entrada del programa (equivale a `main`)	`lobby.launch{start THE GAME} glhf ... gg`	`lobby.launch{start THE GAME} glhf chat.send{"Hola"} gg`
`start THE GAME`	Texto fijo obligatorio del punto de entrada	Dentro de las llaves de `lobby.launch`	`{start THE GAME}`
`glhf`	Abre un bloque («good luck, have fun»)	Después de la cabecera del bloque	`scan x over 0 glhf ... gg`
`gg`	Cierra un bloque («good game»)	Al final del bloque	`scan x over 0 glhf ... gg`
Importaciones			
`travel`	Importa otro archivo del proyecto	`travel archivo from carpeta`	`travel combate from lib`
`from`	Indica la carpeta del archivo importado (`base` = la misma)	`from carpeta`	`travel reglas from base`
Declaración			
`stash`	Declara una variable (guardar en el alijo)	`stash tipo nombre set valor`	`stash score vida set 100`
`hardcoded`	Declara una constante	`hardcoded tipo NOMBRE set valor`	`hardcoded aim GRAVEDAD set 9.8`
`noloot`	Indica que un `combo` no devuelve nada (`void`)	`drops noloot`	`combo saludar{} drops noloot glhf ... gg`
`afk`	Valor nulo («away from keyboard»)	Como valor	`scan jefe matches afk glhf ... gg`
Tipos de datos			

Palabra reservada	Propósito	Forma de uso	Ejemplo
`score`	Número entero	`stash score n set ...`	`stash score oro set 50`
`aim`	Número real	`stash aim n set ...`	`stash aim precision set 0.85`
`lore`	Texto	`stash lore n set ...`	`stash lore nombre set "Link"`
`pixel`	Un carácter	`stash pixel c set ...`	`stash pixel letra set 'A'`
`win`	Valor lógico	`stash win b set ...`	`stash win vivo set hit`
`hit` `miss`	Verdadero y falso	Como valor de tipo `win`	`stash win escudo set miss`
`squad`	Vector de elementos de un mismo tipo	`squad[tipo]`	`stash squad[lore] equipo`
`inventory`	Registro con campos	`inventory Nombre glhf campos gg`	`inventory Item glhf lore nombre gg`
Condicionales			
`scan`	Inicia un condicional (`if`)	`scan condicion glhf ... gg`	`scan vida over 0 glhf ... gg`
`fallback`	Bloque alternativo (`else`)	`gg fallback glhf ... gg`	`gg fallback scan x over 5 glhf ... gg`
`loadout`	Selección múltiple (`switch`)	`loadout valor glhf ramas gg`	`loadout clase glhf ... gg`
`pick`	Caso de un `loadout`	`pick v1 | v2 glhf ... gg`	`pick "mago" glhf ... gg`
`noob`	Caso por defecto (`default`)	`noob glhf ... gg`	`noob glhf poder set 50 gg`
Ciclos			
`farm`	Ciclo con contador (`for`)	`farm inicio | condicion | paso glhf ... gg`	`farm i set 1 | i atmost 10 | i buff glhf ... gg`
`survive`	Ciclo con condición previa (`while`); también cierra el `grind`	`survive condicion glhf ... gg`	`survive vida over 0 glhf ... gg`
`grind`	Ciclo con condición posterior (`do-while`)	`grind glhf ... gg survive{condicion}`	`grind glhf ... gg survive{x under 3}`
`ragequit`	Sale del ciclo (`break`)	Sola, dentro de un ciclo	`scan i matches 8 glhf ragequit gg`
`respawn`	Pasa a la siguiente vuelta (`continue`)	Sola, dentro de un ciclo	`scan i matches 3 glhf respawn gg`
Funciones			

Palabra reservada	Propósito	Forma de uso	Ejemplo
`combo`	Declara una función	`combo nombre{parametros} drops tipo glhf ... gg`	`combo doble{score n} drops score glhf ... gg`
`drops`	Indica el tipo de retorno	`drops tipo`	`drops lore`
`reward`	Devuelve un valor (`return`)	`reward expresion`	`reward n crit 2`
Manejo de errores			
`clutch`	Bloque protegido (`try`)	`clutch glhf ... gg revive {e} glhf ... gg`	`clutch glhf ... gg revive {e} glhf ... gg`
`revive`	Atrapa el error (`catch`)	`revive {nombre} glhf ... gg`	`revive {e} glhf chat.send{e.reason} gg`
`checkpoint`	Bloque que siempre se ejecuta (`finally`)	`gg checkpoint glhf ... gg`	`gg checkpoint glhf chat.send{"fin"} gg`
`wipe`	Lanza un error (`throw`)	`wipe{mensaje}`	`wipe{"Sin energia"}`
Programación orientada a objetos			
`guild`	Declara una clase	`guild Nombre glhf miembros gg`	`guild Heroe glhf ... gg`
`evolves`	Herencia (`extends`)	`guild Hijo evolves Padre`	`guild Mago evolves Heroe glhf ... gg`
`spawn`	Crea un objeto (`new`)	`spawn Clase{argumentos}`	`spawn Heroe{"Ana"}`
`me`	Objeto actual (`this`)	`me.campo`	`me.vida set 100`
`open`	Visibilidad pública	Antes de un campo o `combo`	`open lore nombre`
`party`	Visibilidad protegida	Antes de un campo o `combo`	`party score vida`
`solo`	Visibilidad privada	Antes de un campo o `combo`	`solo score secreto`
Operaciones aritméticas			
`gain`	Suma y concatenación (`+`)	`a gain b`	`"HP: " gain vida`
`drain`	Resta y menos unario (`-`)	`a drain b`, `drain a`	`vida drain 10`
`crit`	Multiplicación (`*`)	`a crit b`	`dano crit 2`
`split`	División (`/`)	`a split b`	`oro split 3`
`scrap`	Residuo (`%`)	`a scrap b`	`n scrap 2`
Operaciones relacionales			
`matches`	Igual (`==`)	`a matches b`	`clase matches "mago"`
`mismatch`	Distinto (`!=`)	`a mismatch b`	`jefe mismatch afk`
`over`	Mayor que (`>`)	`a over b`	`vida over 0`
`under`	Menor que (`<`)	`a under b`	`i under 10`
`atleast`	Mayor o igual (`>=`)	`a atleast b`	`nivel atleast 5`
`atmost`	Menor o igual (`<=`)	`a atmost b`	`n atmost 1`
Operaciones lógicas			

Palabra reservada	Propósito	Forma de uso	Ejemplo
`chain`	Conjunción (`&&`)	`a chain b`	`vivo chain tieneLlave`
`swap`	Disyunción (`||`)	`a swap b`	`esJefe swap esElite`
`dodge`	Negación (`!`)	`dodge a`	`scan dodge vivo glhf ... gg`
Asignación			
`set`	Asigna un valor (`=`)	`variable set valor`	`vida set 100`
`boost`	Suma al valor (`+=`)	`variable boost valor`	`oro boost 50`
`sap`	Resta al valor (`-=`)	`variable sap valor`	`vida sap 10`
`buff`	Incrementa en uno (`++`)	`variable buff`	`nivel buff`
`nerf`	Decrementa en uno (`--`)	`variable nerf`	`vidas nerf`
Reservadas para una versión futura			
`questline`	Interfaces	—	—
`quit`	Terminar el programa	—	—
`server region`	Organización en paquetes	—	—
`stack queue`	Pila y cola	—	—

Los ejemplos de la tabla se muestran en una sola línea por razones de espacio; en código real cada instrucción va en su propia línea, y el contenido de un bloque se escribe entre `glhf` y su `gg`. Las palabras de la última categoría están reservadas para que quien las escriba reciba un mensaje claro (por ejemplo, «GGEZ no tiene ’stack’: la unica coleccion es squad») en lugar de que se interpreten como un nombre cualquiera.

Funciones incorporadas del lenguaje

Además de las palabras reservadas, GGEZ incluye funciones incorporadas, disponibles en cualquier programa sin importar nada. El nombre `chat` está reservado para su módulo: un archivo propio no puede llamarse así.

Función	Propósito	Forma de uso	Ejemplo
`chat.send{x}`	Escribe un valor en la salida estándar, convertido a texto, con salto de línea	`chat.send{expresion}`	`chat.send{"Hola"}`
`chat.rage{x}`	Escribe un valor en la salida de errores; el IDE lo muestra en rojo	`chat.rage{expresion}`	`chat.rage{"Cuidado"}`
`chat.load{}`	Lee una línea escrita por el usuario y la devuelve como `lore`	`chat.load{}`	`stash lore n set chat.load{}`
`v.size`	Cantidad de elementos de un `squad`	`lista.size`	`equipo.size`
`v.recruit{x}`	Agrega un elemento al final de un `squad`	`lista.recruit{valor}`	`equipo.recruit{"Mago"}`
`v.kick{i}`	Quita el elemento de la posición indicada	`lista.kick{posicion}`	`equipo.kick{0}`

El compilador valida estas funciones: `recruit` solo acepta un valor del tipo de los elementos de la lista, `kick` solo acepta un `score`, y `chat.load` no recibe argumentos.

7.3 Sintaxis

Esta sección documenta la sintaxis de GGEZ. En todas las construcciones se aplica la misma regla de bloques: un bloque se abre con `glhf` y se cierra con `gg`, aunque contenga una sola instrucción. La indentación se recomienda por estilo, pero no es obligatoria. La gramática formal completa, en notación BNF extendida, está en el archivo `Traductor/GRAMATICA.md` del repositorio.

7.3.1 Instrucciones

Cada instrucción se escribe en su propia línea; el salto de línea cierra la instrucción y no se usa `;`. Una expresión puede partirse en varias líneas mientras haya un `{` o un `<` sin cerrar.

Declaración de variables. Toda variable se declara con `stash`, su tipo, su nombre y, para los tipos simples, un valor inicial obligatorio con `set`. Una lista puede declararse sin valor y empieza vacía. Las constantes se declaran con `hardcoded`.

```
stash score vida set 100
stash lore nombre set "Link"
stash squad[score] puntos
hardcoded aim GRAVEDAD set 9.8
```

Asignación. Una variable ya declarada recibe un valor nuevo con `set`, sin repetir `stash`. También existen `boost`, `sap`, `buff` y `nerf`.

```
vida set 90
vida sap 15
nivel buff
```

Entrada de datos. `chat.load{}` lee una línea escrita por el usuario en la terminal del IDE. El valor devuelto es siempre un `lore`.

```
stash lore nombre set chat.load{}
```

Salida de datos. `chat.send` imprime un valor o el resultado de una expresión; con `gain` se une texto con números sin convertirlos.

```
chat.send{"Hola mundo"}
chat.send{"Vida: " gain vida}
```

Llamadas a funciones. Una función se invoca con su nombre y los argumentos entre llaves, separados por `|`.

```
saludar{"Ana"}
stash score total set sumar{3 | 4}
```

Importaciones. Se declaran al inicio del archivo, y lo importado se usa con el nombre del módulo delante.

```
travel combate from lib
```

Comentarios. `/w` comenta hasta el final de la línea («whisper», mensaje privado); `/all` … `/end` comenta varias líneas («all chat»).

```
/w Calcula el total del botin
/all
    Este bloque completo
    es un comentario
/end
```

Reglas generales. Se distinguen mayúsculas y minúsculas. Los nombres empiezan con letra o _ y siguen con letras, dígitos o _, sin tildes ni ñ. Los delimitadores cambian respecto a Java: `{ }` agrupan y llaman, `< >` indexan y forman listas, `[ ]` indican el tipo de los elementos de una lista y `|` separa elementos.

7.3.2 Condicionales

GGEZ soporta las tres formas de condicional: simple, con alternativa y múltiple, además de la selección múltiple con `loadout`.

Condicional simple.

```
scan vida over 0 glhf
    chat.send{"Sigue vivo"}
gg
```

Condicional con alternativa. El bloque alternativo se escribe a continuación del `gg` con `fallback`.

```
scan nota atleast 70 glhf
    chat.send{"Aprobado"}
gg fallback glhf
    chat.send{"Reprobado"}
gg
```

Condicional múltiple. Se encadenan ramas con `gg fallback scan`.

```
scan vida over 50 glhf
    chat.send{"Sano"}
gg fallback scan vida over 0 glhf
    chat.send{"Herido"}
gg fallback glhf
    chat.send{"Derrotado"}
gg
```

Selección múltiple. Compara un valor con literales; cada `pick` tiene su propio bloque y puede listar varios valores.

```
loadout clase glhf
    pick "guerrero" glhf
        poder set 80
    gg
```

```
    pick "mago" | "brujo" glhf
        poder set 95
    gg
    noob glhf
        poder set 50
    gg
gg
```

7.3.3 Ciclos

GGEZ incluye tres ciclos: `farm` (equivalente a *for*), `survive` (*while*) y `grind` (*do-while*).

Ciclo farm. Tiene tres partes separadas por `|`: asignación inicial, condición y paso.

```
stash score i set 0
farm i set 1 | i atmost 10 | i buff glhf
    chat.send{i}
gg
```

Ciclo survive. Repite mientras la condición sea `hit`; la condición se evalúa antes de cada vuelta.

```
survive enemigos over 0 glhf
    enemigos nerf
gg
```

Ciclo grind. El cuerpo se ejecuta al menos una vez; la condición va después del `gg`.

```
grind glhf
    intentos buff
gg survive{intentos under 3}
```

7.3.4 Funciones

Toda función se declara con `combo`, su nombre, los parámetros con su tipo entre llaves (separados por `|`) y el tipo de retorno después de `drops`. Las llaves se escriben siempre, aunque no haya parámetros.

Función sin retorno. Se declara con `drops noloot`.

```
combo saludar{lore nombre} drops noloot glhf
    chat.send{"Hola, " gain nombre}
gg
```

Función con retorno. Devuelve un valor con `reward`, que termina la función de inmediato. Todos los caminos deben terminar en `reward`.

```
combo factorial{score n} drops score glhf
    scan n atmost 1 glhf
        reward 1
    gg
    reward n crit factorial{n drain 1}
gg
```

Punto de entrada. Todo programa debe tener exactamente un `lobby.launch{start THE GAME}` en su archivo principal. `start THE GAME` es texto fijo, no son parámetros.

```
lobby.launch{start THE GAME} glhf
    chat.send{"GG EZ"}
gg
```

7.3.5 Operaciones aritméticas

GGEZ usa palabras para las operaciones aritméticas: `gain` (suma), `drain` (resta), `crit` (multiplicación), `split` (división) y `scrap` (residuo). `gain` también une texto y convierte automáticamente el otro operando. `drain` funciona como menos unario. Las llaves `{ }` agrupan para cambiar el orden de evaluación. Dividir entre cero produce un error en ejecución.

```
stash score a set 10
stash score b set 3
chat.send{a gain b}
chat.send{a split b}
stash score r set {a gain b} crit {a drain b}
chat.send{drain r}
```

7.3.6 Operaciones lógicas

Las operaciones lógicas son `chain` (y), `swap` (o) y `dodge` (no). `chain` y `swap` usan cortocircuito, y cada operando debe ser de tipo `win`.

```
scan vida over 0 chain tieneLlave glhf
    chat.send{"Puede abrir la puerta"}
gg
```

7.3.7 Operaciones relacionales

Las comparaciones son `matches`, `mismatch`, `over`, `under`, `atleast` y `atmost`, y devuelven un `win`. `matches` y `mismatch` comparan valores de cualquier tipo, incluido `afk`; las demás solo comparan números. No se encadenan: se escribe `x over 1 chain x under 10`.

```
scan clase matches "mago" glhf
    poder boost 15
gg
```

7.3.8 Clases, registros y errores

Registros. Agrupan campos con nombre; se crean con `spawn` pasando los valores en orden.

```
inventory Item glhf
    lore nombre
    score poder
```

```
gg
```

Clases. Tienen campos con visibilidad, métodos y un constructor (el `combo` que se llama igual que la clase). La herencia se indica con `evolves`.

```
guild Heroe glhf
    party lore nombre

    combo Heroe{lore n} drops noloot glhf
        nombre set n
    gg

    open combo grito{} drops lore glhf
        reward nombre gain ": al ataque!"
    gg
gg
```

Manejo de errores. Un error dentro de `clutch` se atrapa en `revive`; `checkpoint` se ejecuta siempre.

```
clutch glhf
    chat.send{10 split cero}
gg revive {e} glhf
    chat.send{"Linea " gain e.line gain ": " gain e.reason}
gg checkpoint glhf
    chat.send{"Fin del intento"}
gg
```

7.4 Semántica

Esta sección documenta el comportamiento del lenguaje, respondiendo a los aspectos semánticos que estructuran su diseño.

Cómo inicia un programa

Todo programa GGEZ comienza en el bloque de `lobby.launch{start THE GAME}` del archivo principal del proyecto. Al ejecutar, el traductor carga todas las importaciones, analiza el proyecto completo y, si no hay errores, ejecuta las instrucciones de ese bloque en orden. Al terminar el bloque, el programa termina con código de salida 0; un error no atrapado lo termina con código 1.

Cómo se delimitan los bloques

Todo bloque se abre con `glhf` y se cierra con `gg`. El compilador reconoce los bloques por estas palabras y no por el espaciado, de modo que los bloques son siempre visibles e inequívocos.

Si la indentación es obligatoria

No. La indentación no tiene significado para el compilador. Se recomienda indentar cuatro espacios por nivel para facilitar la lectura, y el editor ayuda con colores de sangría y cerrando `gg` automáticamente.

Cómo se declaran y utilizan las variables

Las variables se declaran con `stash`, seguido del tipo, el nombre y el valor inicial. El tipo es explícito y no cambia. Toda variable debe declararse antes de usarse; usar una variable no declarada es un error semántico. Una constante `hardcoded` no puede volver a asignarse. La única conversión implícita es de `score` a `aim`; asignar un `aim` a un `score` es un error. `afk` se puede asignar a variables de tipo `lore`, `pixel`, lista u objeto, pero no a `score`, `aim` ni `win`.

Cómo funcionan los alcances

GGEZ usa alcance de bloque: una variable existe desde su declaración hasta el `gg` del bloque donde se declaró. No se puede declarar dos veces el mismo nombre en un bloque ni ocultar (sombrear) una variable de un bloque exterior, para evitar confusiones. La única excepción es un parámetro o variable local con el nombre de un campo de su clase: el campo se alcanza con `me.campo`.

Cómo se evalúan las expresiones

Las expresiones se evalúan de izquierda a derecha respetando la precedencia de la sección 10.1. `chain` y `swap` usan cortocircuito. Los tipos se verifican antes de ejecutar: una operación con tipos incompatibles es un error semántico. Si los dos operandos de `split` son `score`, el resultado es `score` y se trunca hacia cero.

Cómo funcionan las condiciones y ciclos

Las condiciones deben ser de tipo `win`. En un `scan` se ejecuta solo la primera rama cuya condición sea `hit`. En un `loadout` se ejecuta el primer `pick` que coincide, sin caer en los siguientes, o `noob` si ninguno coincide. `survive` evalúa la condición antes de cada vuelta y `grind` después. `farm` hace la asignación inicial, evalúa la condición antes de cada vuelta y aplica el paso al final de ella. `ragequit` sale del ciclo más cercano y `respawn` salta a la siguiente vuelta; ambos solo son válidos dentro de un ciclo.

Cómo se llaman las funciones

Se evalúan los argumentos de izquierda a derecha, se crea un entorno nuevo con los parámetros y se ejecuta el cuerpo. La cantidad y el tipo de los argumentos deben coincidir con la firma. Los valores simples se pasan por valor y los objetos y listas por referencia. Un `combo` puede llamarse aunque esté definido más abajo, y se permite la recursión hasta 5000 llamadas anidadas. En un método se ejecuta la versión del objeto real (polimorfismo).

Cómo se retornan valores

`reward` devuelve un valor y termina la función de inmediato. El compilador verifica que todo camino de un `combo` con retorno termine en `reward` y que el valor sea del tipo declarado en `drops`. Un `combo drops noloot` no devuelve nada.

Cómo funcionan las importaciones

`travel X from Y` carga `X.ggez` de la carpeta `Y`, relativa al archivo que importa. Todo lo que el archivo declara (`combo`, `guild`, `inventory`) queda disponible con el prefijo del módulo: `X.combo{}`, `X.Tipo`. Cada archivo se carga una sola vez, las importaciones no son transitivas y un archivo importado no puede tener `lobby.launch`.

Cómo se manejan los errores

Los errores se clasifican en léxicos, sintácticos, de importación y semánticos (detectados al compilar) y en ejecución. A ellos se suma el error de proyecto que el IDE reporta si no hay un `.ggez` enfocado ni archivo principal. Cada error indica tipo, archivo, línea, columna, descripción y posible causa. Un programa con errores de compilación no se ejecuta. Los errores en ejecución pueden atraparse con `clutch`/`revive`; si no se atrapan, detienen el programa. El detalle está en la sección 15.

7.5 Tipos de datos

GGEZ soporta cinco tipos simples (`score`, `aim`, `lore`, `pixel` y `win`), el valor nulo `afk` y dos tipos compuestos (`squad` e `inventory`), además de las clases definidas con `guild`. El tipo se declara siempre de forma explícita.

7.5.1 Tipos simples

score. Representa números enteros con signo. Admite todas las operaciones aritméticas y relacionales. Combinado con un `aim`, el resultado es `aim`. La división entre dos `score` descarta la parte decimal (`7 split 2` da `3`).

```
stash score vida set 100
stash score deuda set drain 5
```

aim. Representa números reales. Se escribe con dígitos a ambos lados del punto (`0.5`, `5.0`). Un `score` se convierte automáticamente a `aim` cuando hace falta.

```
stash aim precision set 0.85
stash aim velocidad set 3
```

lore. Representa texto entre comillas dobles. Admite la concatenación con `gain` y las comparaciones `matches` y `mismatch`. Acepta los escapes `\n`, `\t`, `\"` y `\\`. No se indexa por posición.

```
stash lore nombre set "Link"
```

```
stash lore saludo set "Hola, " gain nombre
```

pixel. Representa un solo carácter entre comillas simples.

```
stash pixel inicial set 'L'
```

win. Representa un valor lógico: `hit` (verdadero) o `miss` (falso). Es el tipo de toda condición y de los resultados de las comparaciones. No hay conversión automática desde números.

```
stash win vivo set hit
stash win esJefe set vida over 500
```

afk. Representa la ausencia de valor. Es el valor inicial de los campos de texto, listas y objetos de una clase. Usar un objeto `afk` produce un error en ejecución.

```
stash Heroe jefe set afk
scan jefe matches afk glhf
    chat.send{"Todavia no aparece el jefe"}
gg
```

7.5.2 Tipos compuestos

squad (vector). Lista ordenada de elementos del mismo tipo, indexada desde cero, que crece según se necesite. Se escribe con `< >` y se modifica con `recruit`, `kick` y asignación por índice. Un índice fuera de rango produce un error en ejecución.

```
stash squad[lore] equipo set <"Tanque" | "Sanador">
equipo.recruit{"Arquero"}
equipo<0> set "Guerrero"
equipo.kick{1}
chat.send{equipo.size}
```

inventory (registro). Agrupa campos con nombre y tipo, sin métodos. Se crea con `spawn` pasando los valores en el orden de declaración, y sus campos se leen y modifican con punto.

```
inventory Item glhf
    lore nombre
    score poder
gg

lobby.launch{start THE GAME} glhf
    stash Item espada set spawn Item{"Espada" | 40}
    espada.poder boost 5
    chat.send{espada.nombre gain ": " gain espada.poder}
gg
```

guild (clase). Tipo definido por el usuario con campos, métodos, constructor, visibilidad y herencia simple (sección 8).

Error capturado. La variable de `revive {e}` tiene dos campos de solo lectura: `e.reason` (`lore`) y `e.line` (`score`).

8. Funciones

Las funciones son la unidad principal de organización del código en GGEZ. La sintaxis se documenta en la sección 7.3.4 y su semántica en la sección 7.4. Esta sección consolida las categorías de funciones, las buenas prácticas para su diseño y las restricciones que aplican.

8.1 Categorías de funciones en GGEZ

En GGEZ coexisten tres categorías de funciones:

- **Funciones definidas por el usuario.** Se declaran con `combo` en el nivel superior de un archivo, tienen nombre, parámetros tipados y un tipo de retorno (o `noloot`). Pueden usarse desde otros archivos que importen el suyo con `travel`.
- **Métodos y constructores.** Son `combo` declarados dentro de un `guild`. El que se llama igual que la clase es el constructor, se declara `drops noloot` y se ejecuta al hacer `spawn`. Dentro de un método, `me` se refiere al objeto actual.
- **Funciones incorporadas.** `chat.send`, `chat.rage`, `chat.load` y los métodos de `squad` (`size`, `recruit`, `kick`), disponibles sin importar nada (sección 7.2).

```
guild Personaje glhf
    party lore nombre
    party score vida

    combo Personaje{lore n} drops noloot glhf
        nombre set n
        vida set 100
    gg

    open combo grito{} drops lore glhf
        reward nombre gain ": ..."
    gg
gg

guild Guerrero evolves Personaje glhf
    combo Guerrero{lore n} drops noloot glhf
        nombre set n
        vida set 120
    gg

    combo grito{} drops lore glhf
        reward nombre gain ": POR LA GLORIA!"
    gg
gg

lobby.launch{start THE GAME} glhf
    stash Personaje heroe set spawn Guerrero{"Conan"}
```

```
    chat.send{heroe.grito{}}
gg
```

8.2 Buenas prácticas

- **Funciones breves con propósito único.** Cada `combo` debería hacer una sola tarea identificable.
- **Nombres descriptivos.** Usar verbos que digan qué hace la función (`calcularDano`, `mostrarMenu`, `validarEdad`).
- **Pocos parámetros.** Si una función necesita muchos datos relacionados, conviene agruparlos en un `inventory` o un `guild`.
- **Separar cálculo y presentación.** Las funciones que calculan deberían devolver valores con `reward`, no imprimirlos; la impresión ocurre en `lobby.launch` o en una función dedicada.
- **Modularizar por archivos.** Agrupar en un archivo los `combo` y clases de un mismo tema (por ejemplo, `combate.ggez`, `personajes.ggez`).

8.3 Restricciones del sistema de funciones

- **Argumentos por posición:** no hay argumentos con nombre ni valores por defecto; la cantidad debe coincidir con la de parámetros.
- **Sin sobrecarga:** cada nombre de `combo` es único en su archivo.
- **Sin funciones anónimas:** las funciones no son valores que se puedan guardar en variables.
- **Retorno obligatorio:** todo camino de un `combo` con retorno debe terminar en `reward`.
- **Herencia sin `super`:** el constructor del padre no se ejecuta ni se hereda; el hijo inicializa los campos heredados, que deben ser `party` u `open`. Un método redefinido debe tener la misma firma y no puede reducir su visibilidad.
- **Recursión limitada** a 5000 llamadas anidadas.

Los valores simples (`score`, `aim`, `lore`, `pixel`, `win`) se pasan por valor: modificar el parámetro no afecta la variable original. Las listas, registros y objetos se pasan por referencia: modificar su contenido dentro de la función sí afecta el original.

9. Estructuras de control

La sintaxis de las estructuras de control se documenta en las secciones 7.3.2 y 7.3.3, y su semántica en la sección 7.4. Esta sección consolida las estructuras disponibles y ofrece recomendaciones de estilo.

9.1 Cuadro sinóptico

GGEZ incluye nueve estructuras de control: cuatro de decisión, tres de repetición y dos de manejo de errores.

Estructura	Tipo	Propósito
`scan ... glhf ... gg`	Condicional simple	Ejecuta un bloque solo si se cumple una condición
`scan ... gg fallback glhf ... gg`	Condicional con alternativa	Ejecuta uno de dos bloques según el resultado
`scan ... gg fallback scan ...`	Condicional múltiple	Evalúa varias condiciones y ejecuta la primera verdadera
`loadout ... pick ... noob`	Selección múltiple	Compara un valor con varias opciones
`farm ... | ... | ... glhf ... gg`	Ciclo con contador	Repite un bloque con una variable de control
`survive ... glhf ... gg`	Ciclo con condición previa	Repite mientras se cumpla una condición
`grind glhf ... gg survive{...}`	Ciclo con condición posterior	Repite al menos una vez
`clutch ... revive ... checkpoint`	Manejo de errores	Atrapa un error y continúa
`wipe{...}`	Lanzamiento de error	Produce un error propio

9.2 Recomendaciones de estilo

- **Evitar el anidamiento profundo.** Si un bloque necesita más de dos niveles, conviene extraer parte del código a un `combo`.
- **Preferir salidas tempranas con `reward`.** En una función que clasifica, varios `reward` en condiciones simples se leen mejor que condicionales anidados.
- **Usar `loadout` para comparar un valor con opciones fijas**, en lugar de una cadena larga de `scan`.
- **Elegir el ciclo adecuado.** `farm` cuando se conoce la cantidad de vueltas, `survive` cuando depende de una condición y `grind` cuando el cuerpo debe correr al menos una vez.

9.3 Interrupción de ciclos

GGEZ incluye instrucciones para interrumpir un ciclo: `ragequit` sale del ciclo más cercano (*break*) y `respawn` pasa a la siguiente vuelta (*continue*). Se incluyeron porque son conceptos que el estudiante encontrará en cualquier lenguaje profesional, y sus nombres hacen evidente lo que hacen: abandonar la partida o reaparecer. El compilador verifica que solo se usen dentro de un ciclo.

```
stash score i set 0
farm i set 1 | i atmost 10 | i buff glhf
    scan i matches 3 glhf
        respawn
    gg
    scan i matches 8 glhf
        ragequit
    gg
    chat.send{i}
gg
```

10. Operaciones

GGEZ incluye cuatro familias de operaciones: aritméticas, relacionales, lógicas y de asignación, documentadas en las secciones 7.3.1 y 7.3.5 a 7.3.7. Esta sección presenta la precedencia que aplica cuando se combinan.

10.1 Precedencia de operadores

La siguiente tabla presenta la precedencia de mayor a menor. Los operadores del mismo nivel se evalúan de izquierda a derecha, salvo los unarios `dodge` y `drain`. Los relacionales no se encadenan.

Nivel	Operadores	Tipo
1 (mayor)	`{ }`, `.`, `< >`	Agrupación, llamada, acceso a miembro e índice
2	`dodge`, `drain` (unario)	Negación lógica y menos unario
3	`crit`, `split`, `scrap`	Multiplicación, división y residuo
4	`gain`, `drain`	Suma (y concatenación) y resta
5	`matches`, `mismatch`, `over`, `under`, `atleast`, `atmost`	Relacionales
6	`chain`	Conjunción lógica
7	`swap`	Disyunción lógica
8 (menor)	`set`, `boost`, `sap`	Asignación (solo como instrucción)

10.2 Uso de llaves para agrupar

En GGEZ las llaves `{ }` cumplen el papel de los paréntesis: agrupan una expresión para cambiar el orden de evaluación. No se confunden con una llamada por su posición: después de un nombre son una llamada (`chat.send{"hola"}`); donde se espera un valor son una agrupación. Se recomienda usarlas aunque no sean necesarias cuando aclaran la intención.

```
stash score r1 set 2 gain 3 crit 4       /w 14: crit primero
stash score r2 set {2 gain 3} crit 4     /w 20: las llaves primero
```

10.3 Evaluación de cortocircuito

En `A chain B`, si `A` es `miss`, `B` no se evalúa; en `A swap B`, si `A` es `hit`, `B` no se evalúa. Esto permite proteger operaciones que fallarían:

```
scan jefe mismatch afk chain jefe.vida over 0 glhf
    chat.send{"El jefe sigue en pie"}
gg
```

11. Organización de archivos

Un proyecto GGEZ es una carpeta con uno o más archivos `.ggez`, que pueden organizarse en subcarpetas. Esta sección describe su estructura y las reglas para nombrar y organizar los archivos.

11.1 Estructura de un proyecto

Todo proyecto tiene un archivo principal, el único con `lobby.launch`. Los demás son bibliotecas con `combo`, `guild` e `inventory`, importadas con `travel`.

```
MiAventura/
    .ggezproyecto           (guarda cual es el archivo principal)
    principal.ggez          (contiene lobby.launch)
    reglas.ggez             (combos de las reglas del juego)
    lib/
        personajes.ggez     (guilds e inventories)
```

`travel` busca el archivo en una carpeta relativa al que importa (`from base` es la misma carpeta; `from lib`, la subcarpeta `lib`), de modo que un proyecto puede organizarse en carpetas.

11.2 Gestión desde el aplicativo

El árbol de archivos del IDE permite crear un proyecto, abrir una carpeta existente, crear archivos y carpetas, abrir archivos en pestañas, renombrarlos, eliminarlos y designar el archivo principal con clic derecho, «Marcar como principal». El archivo principal se identifica en el árbol con la etiqueta «(principal)», y la designación se guarda en el archivo `.ggezproyecto` de la raíz del proyecto. El archivo activo se identifica por su pestaña y por la ruta que se muestra sobre el editor; Compilar y Ejecutar usan ese archivo si es un `.ggez`, y el principal solo cuando no lo es. Las pestañas abiertas se conservan al cerrar y volver a abrir el IDE.

11.3 Validaciones del archivo principal

Antes de compilar o ejecutar un proyecto se verifica que:

- Si la pestaña enfocada no es un `.ggez`, exista un archivo designado como principal y que no haya sido borrado. Si no, el IDE reporta un **error de proyecto** que indica cómo designarlo.
- El archivo principal tenga exactamente un `lobby.launch{start THE GAME}`.
- Todas sus importaciones sean válidas (sección 12).

Si el archivo que se ejecuta está en una carpeta con `nivel.json`, se trata como un nivel del modo de juego y se ejecutan las pruebas del nivel (sección 16).

11.4 Convenciones de nomenclatura

Se recomienda que los nombres de archivo usen solo letras, dígitos y guiones bajos, sin espacios ni tildes, y que describan su contenido (`combate.ggez` en lugar de `archivo1.ggez`). El archivo principal puede llamarse `principal.ggez` o como el proyecto. La extensión `.ggez` es obligatoria. Como `travel` recibe el nombre del archivo como un identificador, un archivo cuyo nombre tenga espacios, tildes o guiones no puede importarse, y no puede llamarse `chat`.

12. Importaciones

El sistema de importaciones permite reutilizar funciones y tipos definidos en otros archivos del mismo proyecto. La sintaxis y la semántica se documentan en las secciones 7.3.1 y 7.4.

12.1 Forma de importación

GGEZ tiene una sola forma de importación: `travel archivo from carpeta`. Todo lo que el archivo declara queda disponible con el nombre del módulo como prefijo, igual que `chat.send`. Así siempre se sabe de qué archivo viene cada cosa.

Archivo `lib/personajes.ggez`:

```
guild Heroe glhf
    open lore nombre

    combo Heroe{lore n} drops noloot glhf
        nombre set n
    gg
gg

combo saludo{Heroe h} drops lore glhf
    reward "Hola, " gain h.nombre
gg
```

Archivo `principal.ggez`:

```
travel personajes from lib

lobby.launch{start THE GAME} glhf
    stash personajes.Heroe h set spawn personajes.Heroe{"Ana"}
    chat.send{personajes.saludo{h}}     /w imprime: Hola, Ana
gg
```

12.2 Validaciones del compilador

El compilador realiza las siguientes validaciones sobre cada importación. Todas se reportan como `error de importacion`:

Validación	Descripción
Existencia del archivo	El archivo indicado en `travel` debe existir en la carpeta indicada, con extensión `.ggez`. Mensaje: `no existe tienda.ggez en la carpeta 'lib'`.
Existencia de la función o el tipo	Lo que se usa con el prefijo del módulo debe estar declarado en ese archivo. Mensaje: `el modulo combate no tiene 'curar'. Tiene: danoTotal`.

Validación	Descripción
Cantidad de argumentos	Cada llamada a un `combo` importado debe pasar tantos argumentos como parámetros declara.
Tipos de los argumentos	Cada argumento debe ser del tipo del parámetro. Mensaje: `el parametro 'base' de 'danoTotal' es score y se paso un lore`.
Módulos duplicados	No se puede importar dos veces el mismo módulo en un archivo, ni dos módulos distintos con el mismo nombre, ni un módulo llamado `chat`.
Importaciones circulares	Si `a` importa `b` y `b` importa `a`, directa o indirectamente, se reporta el ciclo completo: `importacion en circulo: mapa.ggez -> zona.ggez -> mapa.ggez`.
Errores en el archivo importado	Si un archivo importado tiene errores léxicos, sintácticos o semánticos, la compilación se detiene y el error se reporta con su categoría original, el nombre del archivo importado y su línea.

Las importaciones no son transitivas: para usar algo de un módulo hay que importarlo en el mismo archivo. Los archivos del proyecto que no se alcanzan desde el archivo principal no se analizan.

12.3 Restricciones estructurales

Los archivos importados solo pueden contener `travel`, `combo`, `guild` e `inventory`. No pueden tener `lobby.launch`; si lo tienen, el compilador reporta: `un archivo que se importa con travel es una biblioteca: no puede tener lobby.launch`. Las importaciones van al inicio del archivo, y cada archivo se carga una sola vez aunque varios lo importen.

13. Proceso de compilación

El proceso de compilación de GGEZ consiste en cuatro fases que analizan el código fuente, a partir del archivo principal, para detectar cualquier error antes de permitir su ejecución. Esta sección describe las fases del compilador y las herramientas que lo implementan.

13.1 Fases del compilador

1. Análisis léxico (`lexer.py`). El código fuente se convierte en una secuencia de tokens: palabras reservadas, identificadores, literales y símbolos. Se descartan los comentarios y se emite un fin de línea por cada salto, excepto dentro de un `{` o un `<` sin cerrar. Se detectan errores léxicos como caracteres no reconocidos, cadenas sin cerrar o nombres con tilde.

2. Análisis sintáctico (`sintactico.py`). Un analizador descendente recursivo, con una función por cada regla de la gramática, organiza los tokens en un árbol sintáctico (`arbol.py`). Se detectan errores sintácticos como bloques sin `gg`, palabras fuera de orden, un `combo` sin `drops` o expresiones incompletas.

3. Resolución de importaciones (`cargador.py`). A partir del archivo principal se sigue cada `travel` en profundidad: cada archivo importado se analiza léxica y sintácticamente, se verifica que no tenga `lobby.launch` y que no se repita un módulo, y se detectan las importaciones circulares llevando el camino de archivos que se están cargando. Cada archivo se carga una sola vez.

4. Análisis semántico (`semantico.py`). Sobre los árboles de todos los archivos, con una tabla de símbolos por alcance, se verifican: el punto de entrada único, las declaraciones y redeclaraciones, el sombreado, los tipos de cada expresión, asignación y retorno, la cantidad y el tipo de los argumentos (también de los `combo` importados), la visibilidad y la herencia de las clases, las constantes, que todo camino de un `combo` con retorno termine en `reward` y que `ragequit` y `respawn` estén dentro de un ciclo. En esta fase también se calculan las advertencias.

13.2 Herramientas técnicas

El compilador está escrito en Python 3, sin bibliotecas externas: el analizador léxico, el sintáctico, el cargador de importaciones y el analizador semántico se implementaron a mano. Se eligió escribir el analizador sintáctico como descendente recursivo porque cada función corresponde a una regla de la gramática de `GRAMATICA.md`, lo que facilita leerlo, mantenerlo y producir mensajes de error propios en español. Las palabras reservadas están definidas en un único archivo (`tokens.py`), de modo que cambiar una palabra del lenguaje se hace en un solo lugar.

El compilador se usa desde el IDE o desde la línea de comandos:

Comando	Uso
`ggezc compile <archivo>`	Valida el proyecto completo (fases 1 a 4) sin ejecutar.
`ggezc run <archivo>`	Valida y, si no hay errores, ejecuta.

Comando	Uso
`ggezc check <archivo>`	Corre las pruebas del `nivel.json` de la carpeta (modo de juego).
`ggezc tokens <archivo>`	Muestra los tokens del análisis léxico.
`--pasos N`	Cambia la cantidad de maná (por defecto 5 000 000).
`--eventos`	Emite las líneas `@ggez` que el IDE usa para el modo de juego.

```
$ python ggezc.py compile principal.ggez
Compilacion exitosa: principal.ggez y 2 archivo(s) importado(s)
```

13.3 Reporte de errores

Cuando el compilador detecta un error en cualquiera de las fases, la compilación se detiene y se reporta ese único error. Al corregirlo y volver a compilar aparece el siguiente, si lo hay. Así el estudiante se concentra en un problema a la vez, en lugar de una lista larga de errores que muchas veces son consecuencia del primero. El error se escribe en la salida de errores con el formato estándar (sección 15) y el IDE lo muestra en la pestaña Problemas. Si el proyecto compila, se reportan además sus advertencias, que no impiden ejecutar.

14. Proceso de ejecución

Una vez que la compilación termina sin errores, el intérprete ejecuta el programa. Esta sección describe la estrategia del intérprete y cómo gestiona el estado del programa.

14.1 Estrategia general

El intérprete de GGEZ (`evaluador.py`) recorre directamente el árbol sintáctico y ejecuta cada nodo según su tipo (*tree-walking interpreter*). No genera código intermedio ni usa `eval` o `exec` de Python. Este enfoque es sencillo de implementar y adecuado para el propósito educativo del lenguaje.

14.2 Flujo de ejecución

- Al presionar Ejecutar (`F5`), el IDE guarda todos los archivos y ejecuta el archivo `.ggez` de la pestaña enfocada; si la pestaña enfocada no es un `.ggez`, usa el archivo principal del proyecto. Si el archivo está en una carpeta de nivel, corre las pruebas del nivel en vez de ejecutarlo.
- El IDE lanza `ggezc run` en una terminal integrada libre, con la carpeta del proyecto como directorio de trabajo.
- El traductor realiza las cuatro fases de compilación. Si hay un error, lo reporta y termina con código 1 sin ejecutar nada.
- El intérprete registra los `combo`, `guild` e `inventory` de todos los archivos y ejecuta el bloque de `lobby.launch` instrucción por instrucción.
- Las declaraciones y asignaciones actualizan el entorno; las estructuras de control deciden la siguiente instrucción; las llamadas crean un entorno nuevo y transfieren el control al cuerpo de la función. `reward`, `ragequit`, `respawn` y `wipe` se implementan como señales internas que suben hasta la construcción que las maneja.
- `chat.send` escribe en la salida estándar, `chat.rage` en la de errores y `chat.load` lee de la entrada estándar. La terminal del IDE está conectada a los tres, por lo que el estudiante escribe sus respuestas directamente en ella.
- La ejecución termina al completar el bloque de `lobby.launch`, con un error no atrapado, al agotarse el maná o cuando el usuario la interrumpe con `Ctrl+C` en la terminal. Al terminar, el IDE actualiza el maná, las misiones y los logros.

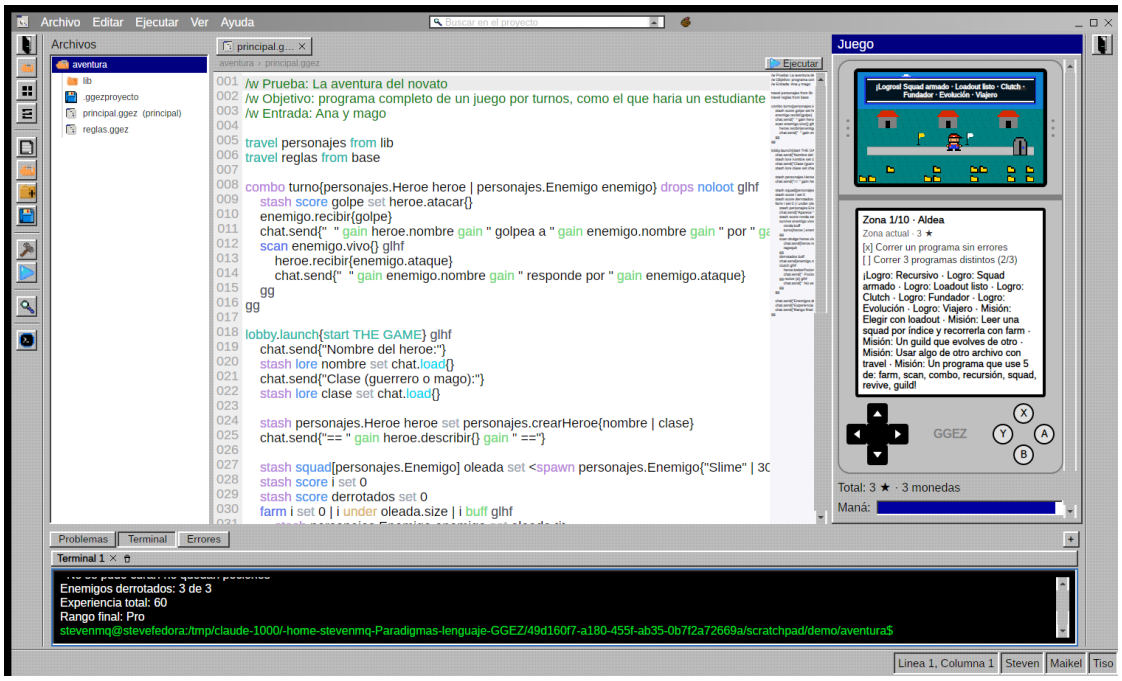


Figura 1: Ejecución de un programa con entrada interactiva en la terminal integrada.

14.3 Gestión de alcances

Existe un entorno global con las funciones y tipos de cada archivo. Cada bloque (rama de un `scan`, cuerpo de un ciclo, bloque de `clutch`) crea un entorno hijo que se descarta al llegar a su `gg`. Cada llamada a un `combo` crea un entorno nuevo con sus parámetros, que no ve las variables de quien hizo la llamada. Dentro de un método, los nombres se buscan primero en las variables locales, después en los campos y métodos del objeto y por último en los `combo` globales.

14.4 Ejecución de importaciones

Cuando el intérprete encuentra una llamada a un `combo` importado (`modulo.combo{}`), ejecuta el cuerpo de esa función con su propio entorno, aislado del archivo que la llamó, y devuelve el valor al punto de la llamada. Lo mismo ocurre con `spawn modulo.Tipo{}`. Los archivos importados no ejecutan código propio: solo aportan declaraciones.

14.5 Limitaciones de la ejecución

- **Maná.** Cada instrucción y cada vuelta de un ciclo gasta un punto. El programa empieza con 5 000 000; si se agota, termina con el error «sin mana». Así un ciclo infinito no congela el IDE. Este error no se puede atrapar con `revive`.
- **Recursión.** La profundidad máxima es de 5000 llamadas anidadas; al superarla se produce un error que indica revisar el caso base.
- **Velocidad.** Un intérprete que recorre el árbol es más lento que un lenguaje compilado a código nativo, lo cual es aceptable para el propósito educativo del lenguaje.

15. Manejo de errores

El sistema de manejo de errores es uno de los componentes centrales de GGEZ, porque es lo primero que encuentra un principiante cuando algo sale mal. Esta sección documenta las categorías de errores, el formato de los mensajes y el comportamiento del aplicativo.

15.1 Categorías de errores

Categoría	Momento de detección	Ejemplos típicos
Léxicos	Análisis léxico (fase 1)	Caracteres no reconocidos, cadenas sin cerrar, nombres con tilde
Sintácticos	Análisis sintáctico (fase 2)	Bloques sin `gg`, `combo` sin `drops`, punto de entrada mal escrito, falta un valor
De importación	Resolución de importaciones (fase 3) y llamadas a lo importado (fase 4)	Archivo o función inexistente, módulo duplicado, importación circular, argumentos incorrectos en una llamada importada
Semánticos	Análisis semántico (fase 4)	Variable no declarada, tipos incompatibles, cantidad de argumentos, redeclaración, sin punto de entrada
En ejecución	Interpretación del programa	División entre cero, índice fuera de rango, objeto `afk`, `wipe` no atrapado, sin maná, recursión excesiva
De proyecto	En el IDE, antes de compilar	La pestaña enfocada no es un `.ggez` y no hay archivo principal, o este fue borrado

15.2 Formato estándar de los mensajes

Cada mensaje de error incluye: (1) el archivo, (2) la línea, (3) la columna, (4) el tipo de error, (5) una descripción del problema y (6) una posible causa. Se escribe en dos líneas:

```
principal.ggez:7:25: error semantico: 'escudo' no esta declarada
    posible causa: Declarela con stash antes de usarla, o revise si
    escribio mal el nombre o si quedo en otro bloque.
```

Para encontrar la posible causa, el traductor busca en el mensaje fragmentos conocidos (por ejemplo «no esta declarada» o «se obtuvo») y, si ninguno coincide, usa una sugerencia general según el tipo de error. Los mensajes evitan tildes para que se vean bien en cualquier consola.

En el IDE, la pestaña Problemas presenta cada error con su ícono, el archivo y la línea, la descripción y, debajo, la posible causa en gris.

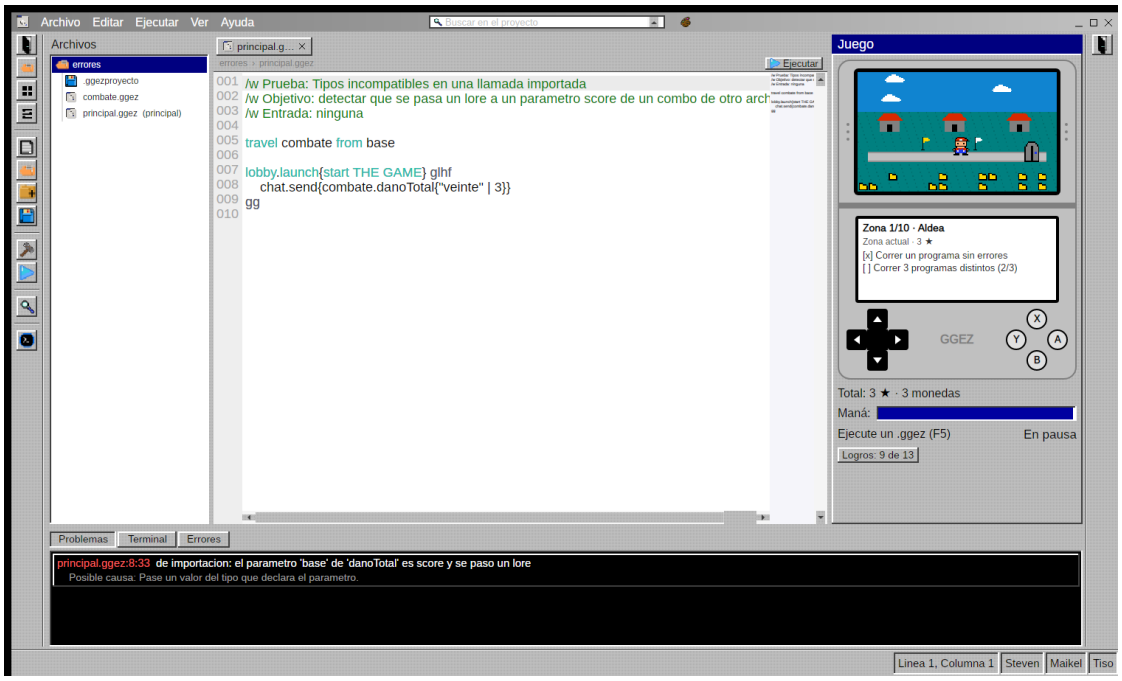


Figura 2: Pestaña Problemas con un error y su posible causa.

15.3 Filosofía de retroalimentación

Los mensajes de GGEZ siguen cuatro principios: (1) **lenguaje claro**, en español y sin jerga innecesaria; (2) **ubicación precisa**, con archivo, línea y columna; (3) **orientación a la solución**, diciendo qué revisar y no solo qué falló; y (4) **un error a la vez**, para no abrumar.

15.4 Comportamiento del aplicativo

Un programa con errores de compilación no se ejecuta. Los errores en ejecución detienen el programa en el punto donde ocurren, salvo que se atrapen con `clutch`/`revive`. En ambos casos la terminal muestra el error en rojo y la pestaña Problemas lo lista; un clic sobre el problema abre el archivo y lleva el cursor a la línea. La pestaña Errores (menú Ejecutar, «Ver errores») conserva los mensajes del IDE. Cada vez que se compila o ejecuta, la pestaña Problemas se limpia y muestra solo el estado actual.

15.5 Advertencias

Cuando el programa compila sin errores, el compilador revisa además tres situaciones que no impiden ejecutar pero suelen indicar un descuido:

Advertencia	Cuándo se reporta
Variable sin usar	Una variable declarada cuyo valor nunca se lee. Asignarle un valor no cuenta como leerla.

Advertencia	Cuándo se reporta
`combo` sin usar	Un `combo` del archivo principal que nunca se llama; las llamadas a sí mismo no cuentan. Los archivos importados no se revisan, porque pueden tener funciones que el programa no necesita.
Código que nunca se ejecuta	Instrucciones escritas después de un `reward`, `ragequit`, `respawn` o `wipe` en el mismo bloque.

Las advertencias usan el mismo formato que los errores, con el tipo `advertencia`, y el IDE las muestra en amarillo.

```
principal.ggez:6:5: advertencia: la variable 'oro' se declaro pero su valor nunca
    se usa
```

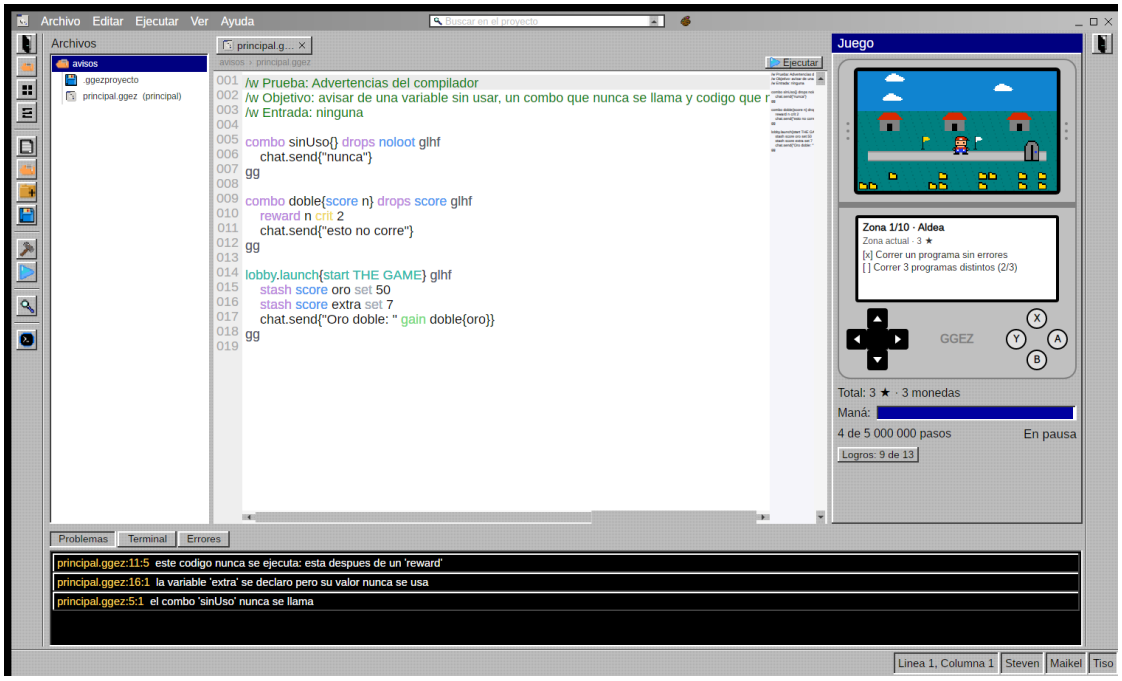


Figura 3: Advertencias en amarillo en la terminal y en la pestaña Problemas.

16. Innovación

La característica de innovación de GGEZ es que **el entorno de desarrollo es en sí mismo un juego**, cuyo progreso depende de que el código del estudiante funcione de verdad. Se complementa con una terminal real con asistente de inteligencia artificial y un sistema de extensiones. Esta sección describe su funcionamiento, cómo se integra con el traductor y por qué se eligió.

16.1 Descripción funcional

El IDE tiene un panel de juego, a la derecha del editor, que muestra el maná, el mapa del mundo, el nivel actual, las monedas, las pistas y los logros. Cada vez que el estudiante ejecuta un programa, el traductor informa al IDE cuánto maná gastó, qué construcciones del lenguaje se ejecutaron y si obtuvo algún logro; el IDE actualiza el panel con esa información. Nada se gana con solo escribir código: solo cuenta lo que se ejecutó sin errores.

16.2 Componentes

Maná. Cada instrucción y cada vuelta de ciclo gasta un punto de maná, y la barra muestra cuánto gastó el programa. Además de impedir que un ciclo infinito congele el IDE, hace visible el costo de un algoritmo: un programa más eficiente gasta menos.

Mundo y misiones. Un mapa con diez zonas en orden: Aldea, Pradera, Bosque de los ciclos, Río, Cueva, Torre, Mercado, Castillo, Gremio y Guarida del jefe. Cada zona tiene misiones que se cumplen ejecutando programas que usen ciertas construcciones (por ejemplo, un programa con `farm`, `survive` y `grind`, o uno con un `guild` que `evolves` de otro). Al completar las misiones de una zona se abre la siguiente.

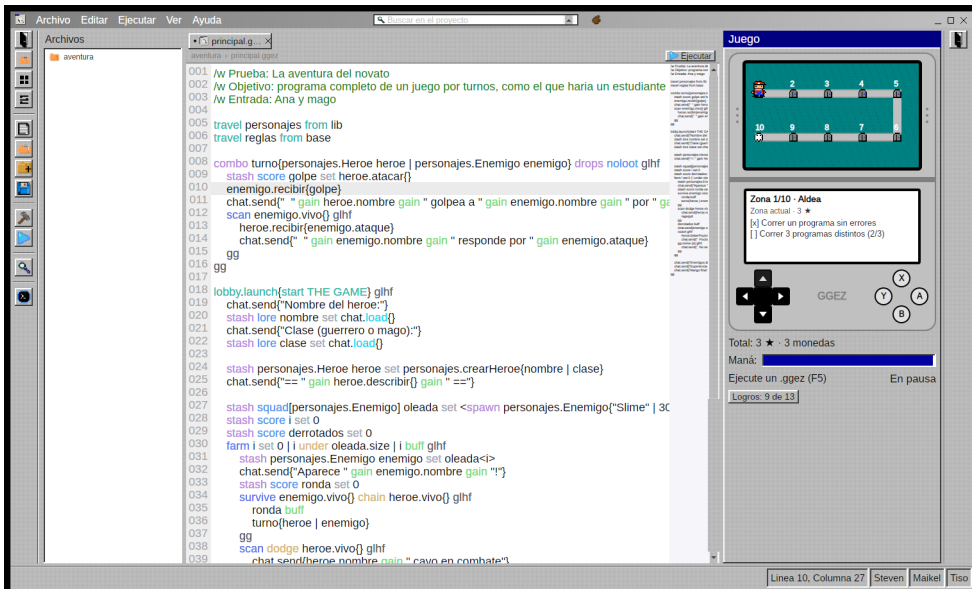


Figura 4: Mapa del mundo con la zona actual y sus misiones.

Niveles y estrellas. Una carpeta con un archivo `nivel.json` define un reto: título, pruebas (una llamada

o una entrada, con su salida esperada), pistas y una meta de pasos. Al ejecutar, el traductor corre las pruebas y el IDE otorga hasta tres estrellas: una por pasar todas las pruebas, otra por lograrlo al primer intento y otra por no superar la meta de pasos. Cualquier docente puede crear niveles nuevos con un archivo de texto, sin tocar el código del IDE.

```
{
  "titulo": "Suma de Gauss",
  "meta_pasos": 5,
  "pistas": ["Con un farm de 1 a n y boost se pasan las pruebas", "..."],
  "pruebas": [
    { "llamada": "sumarHasta{10}", "salida": "55" },
    { "llamada": "sumarHasta{0}",  "salida": "0" }
  ]
}
```

Monedas y pistas. Cada estrella ganada da una moneda, y cada pista de un nivel cuesta una. El estudiante decide si pide ayuda o lo intenta solo.

Logros. Trece logros reconocen hitos de aprendizaje: correr el primer programa sin errores, declarar un `combo`, escribir una recursión, usar `squad`, usar `loadout`, usar los tres ciclos en un programa, atrapar un error, declarar un `guild`, usar herencia, importar un archivo, quedarse sin maná, ganar tres estrellas en un nivel y comprar una pista. Al obtener uno, el IDE muestra una notificación.

Temas desbloqueables. Las estrellas desbloquean temas visuales para el editor (con 6, 15 y 25 estrellas), además de un modo oscuro disponible desde el inicio.

Terminal real con asistente de inteligencia artificial. El segmento de salida admite hasta cuatro terminales en columnas, con comandos propios (`cd`, `clear`, `exit`, `help`). Al instalar la extensión de Claude, las terminales pasan a ser terminales reales del sistema operativo, capaces de correr cualquier programa interactivo, entre ellos el asistente `claude`. Así el estudiante puede pedir, dentro del mismo IDE, que le expliquen un error o un concepto.

Sistema de extensiones. Desde la vista de extensiones se instalan y desinstalan complementos: sangría de colores, resaltado de espacios al final de la línea, resaltado de `TODO`, colores por nivel de bloques, modo oscuro, bloques al estilo Pac-Man, la terminal de Claude y los temas desbloqueables.

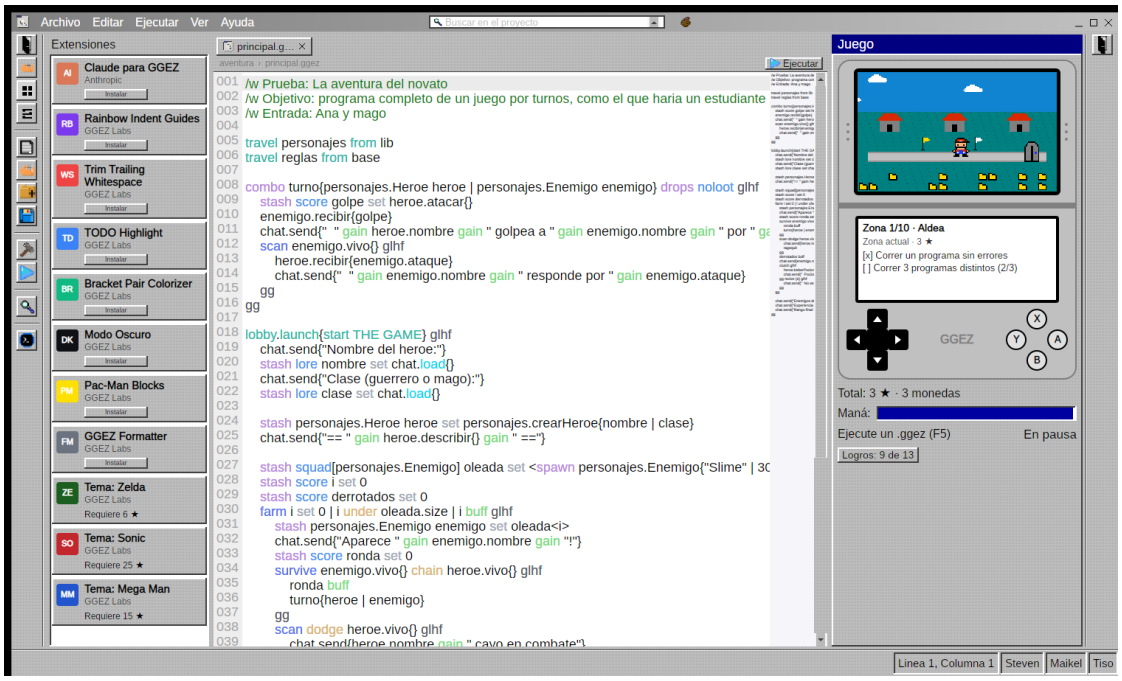


Figura 5: Vista de extensiones.

16.3 Funcionamiento técnico

El IDE ejecuta el traductor con la opción `--eventos`. Al terminar el programa, el traductor escribe en la salida líneas que empiezan con `@ggez`, que el IDE no muestra en la terminal sino que envía al panel de juego:

Evento	Significado
`@ggez pasos <usados> <limite>`	Maná gastado y maná disponible.
`@ggez logro <id>`	El programa cumplió las condiciones de un logro.
`@ggez programa <huella> <usos>`	Huella del programa y construcciones que se ejecutaron (ciclos, recursión, clases, importaciones...), para avanzar las misiones.
`@ggez nivel <pasadas> <total> <pasos> <meta> <titulo>`	Resultado de las pruebas de un nivel, para calcular las estrellas.

Las construcciones usadas se registran durante la ejecución, no al leer el código: un `farm` que nunca dio una vuelta no cuenta. La huella identifica el programa, de modo que una misión que pide varios programas distintos no se cumple ejecutando el mismo dos veces. El progreso (estrellas, monedas, logros, misiones y récords de cada nivel) se guarda en el archivo `progreso.json` de la carpeta de configuración del usuario.

16.4 Validaciones e integración

- El archivo `nivel.json` puede venir de otra persona, por lo que el traductor revisa su forma antes de usarlo (título de texto, meta de pasos entera y positiva, al menos una prueba con su salida, llamadas de una sola línea) y reporta un error claro si algo está mal.
- Un evento con un identificador que el IDE no conoce se ignora.
- Solo se puede comprar una pista si hay monedas suficientes y quedan pistas en el nivel.
- La gamificación no requiere configuración: funciona desde la primera ejecución y no interfiere con el programa, cuya salida se ve completa en la terminal.

16.5 Justificación

La elección de esta característica responde a tres consideraciones. Primero, atiende directamente el problema planteado: el estudiante de primer ingreso conoce los videojuegos y su sistema de progreso, y el IDE usa ese mismo lenguaje para mostrarle que está avanzando. Segundo, está ligada al aprendizaje real, porque solo avanza con código que se ejecuta y premia construcciones concretas del lenguaje, por lo que no es un adorno separado de la programación. Tercero, es demostrable durante la presentación: basta ejecutar un programa para ver cómo cambia el maná, se cumple una misión o aparece un logro.

17. Arquitectura del aplicativo

El aplicativo tiene dos componentes que se comunican mediante procesos: el **IDE**, escrito en C# con Avalonia, y el **traductor**, escrito en Python. Esta sección describe las tecnologías utilizadas, la organización en capas y los módulos del código fuente.

17.1 Tecnologías utilizadas

- **C# y .NET.** Lenguaje y plataforma del IDE.
- **Avalonia.** Biblioteca de interfaz gráfica multiplataforma que permite que el mismo IDE funcione en Linux y Windows.
- **CommunityToolkit.Mvvm.** Implementa el patrón Modelo-Vista-VistaModelo con propiedades observables y comandos.
- **RoyalTerminal.** Control de terminal que usa la extensión de Claude para ofrecer terminales reales.
- **Python 3.** Lenguaje del traductor, sin bibliotecas externas.
- **Python portátil (python-build-standalone).** Intérprete de Python que se incrusta en el ejecutable para que el usuario no tenga que instalarlo.
- **Nix.** El repositorio incluye un `flake.nix` para preparar el entorno de desarrollo.
- **Git y GitHub.** Control de versiones y trabajo en equipo.

17.2 Organización en capas

Capa de presentación (IDE). Contiene todo lo visual: ventana principal, menús, árbol de archivos, editor, terminales, pestañas de errores y problemas, panel de juego, ayuda, extensiones y ajustes. Sigue el patrón MVVM: las vistas (`.axaml`) solo muestran datos y los modelos de vista (`ViewModels`) contienen la lógica de la interfaz.

Frontera de integración. El IDE y el traductor solo se comunican por texto: el IDE lanza `ggezc` como un proceso, le pasa la entrada del usuario y lee su salida. Los errores tienen un formato fijo (`archivo:linea:columna: tipo: mensaje`) que el IDE interpreta con una expresión regular, y los eventos del juego empiezan con `@ggez`. Gracias a esto, el traductor puede usarse y probarse sin el IDE.

Capa de lógica del lenguaje (traductor). Contiene el compilador, el intérprete, las pruebas de niveles y los eventos del juego. No depende de la interfaz gráfica.

Capa de servicios. En el IDE, componentes de apoyo: diálogos, configuración, persistencia de la sesión y de las extensiones, temas, progreso del juego, procesos de las terminales e instalación del traductor incrustado.

17.3 Estructura de módulos

```
ParadigmasLenguajePropio/src/ParadigmasLenguajePropio/   (IDE)
    Views/          ventana, menu, barra de estado, explorador, extensiones,
                    ajustes, editor, terminal, panel de juego
    ViewModels/     logica de cada vista (abrir, guardar, ejecutar, errores)
    Models/         archivos del arbol, problemas, logros, mundo, ayuda
    Services/       dialogos, sesion, temas, progreso, terminales, instalacion
    Controls/       editor (coloreado, autocompletado, minimapa), arbol
    Themes/, Assets/   estilos, fuentes e iconos
Traductor/                                              (traductor)
    ggezc.py        punto de entrada de la linea de comandos
    GRAMATICA.md    gramatica formal
    src/ggez/
        cli.py          comandos run, compile, check, tokens
        tokens.py       palabras reservadas
        lexer.py        analisis lexico
        sintactico.py   analisis sintactico
        arbol.py        nodos del arbol sintactico
        cargador.py     importaciones con travel
        semantico.py    analisis semantico y advertencias
        evaluador.py    interprete
        errores.py      formato de errores y posible causa
        nivel.py        pruebas de nivel.json
        logros.py       eventos del modo de juego
    pruebas/        casos de la seccion 18 y correr.py
    ejemplos/       programas de ejemplo
publicar.sh         genera el ejecutable unico
```

17.4 Flujo de una operación típica

Al presionar Compilar (`F6`), el IDE guarda todos los archivos, limpia la pestaña Problemas, toma el archivo `.ggez` enfocado (o el principal, si no hay uno) y lanza `ggezc compile` en una terminal integrada. El traductor ejecuta las cuatro fases de la sección 13 y escribe «Compilacion exitosa» con sus advertencias, o un único error. El IDE lee la salida de errores línea por línea, colorea cada línea según su tipo y llena la pestaña Problemas. Al ejecutar (`F5`) el flujo es el mismo con `ggezc run --eventos`: la salida del programa aparece en la terminal, la entrada del usuario se envía al proceso y, al terminar, los eventos `@ggez` actualizan el panel de juego.

17.5 Distribución

El entregable es un ejecutable único por sistema operativo, generado con el script `publicar.sh` de la raíz del repositorio (`./publicar.sh linux`, `windows` o `todo`). El script publica el IDE como un solo archivo y le incrusta un Python portátil y el traductor; la primera vez que se abre, el IDE los despliega en la carpeta de datos del usuario. Así el estudiante no necesita instalar .NET ni Python.

17.6 Consideraciones de diseño

La separación entre IDE y traductor fue la decisión principal de la arquitectura: permitió que los integrantes trabajaran en paralelo, que el traductor se probara de forma automática sin interfaz y que el formato de errores fuera un contrato simple entre ambos. El patrón MVVM mantiene la interfaz separada de su lógica, y concentrar las palabras reservadas en un solo archivo hace que el lenguaje pueda ajustarse sin tocar el resto del traductor.

18. Pruebas

Esta sección documenta la batería de pruebas realizadas sobre GGEZ para verificar su funcionamiento. Las pruebas se organizan según las categorías establecidas por el enunciado del proyecto, y cada una indica su objetivo, entrada, resultado esperado, resultado obtenido y estado (aprobada o reprobada).

Cada caso es un proyecto GGEZ real guardado en el repositorio, dentro de una carpeta por categoría en `Traductor/pruebas/`: `1_validos/`, `2_sintacticos/`, `3_semanticos/`, `4_importaciones/`, `5_circular/`, `6_completa/` y `7_advertencias/`. Cada caso tiene su `principal.ggez` (con una cabecera de comentarios que documenta la prueba, su objetivo y su entrada), un archivo `esperado.txt` con la salida esperada completa (salida estándar, salida de errores y código de salida) y, si el programa lee datos, un archivo `entrada.txt`.

El script `pruebas/correr.py` ejecuta cada caso con el traductor real y compara la salida obtenida con la esperada; se ejecuta con `make test` desde la carpeta `Traductor`. Las tablas de esta sección se generaron con ese mismo script (`python pruebas/correr.py --latex`), por lo que el resultado obtenido corresponde a una ejecución real. Los números de línea de los mensajes corresponden al archivo del caso, incluida su cabecera de comentarios. Las 23 pruebas resultaron aprobadas.

Categoría	**Casos**	**Aprobados**
Programas válidos	5	5
Errores sintácticos	5	5
Errores semánticos	5	5
Importaciones	5	5
Importación circular	1	1
Prueba completa relacionada con el problema	1	1
Advertencias	1	1
Total	**23**	**23**

18.1 Programas válidos

Casos de la carpeta `pruebas/1_validos/`.

Prueba y objetivo	**Entrada**	**Resultado esperado**	**Resultado obtenido y estado**

1. Tipos simples y operaciones. Declarar los cinco tipos simples y una constante, y usar operaciones aritméticas, relacionales y lógicas	`1_validos/01_tipos_y_operaciones`. Entrada: ninguna	`Novato (rango B)`
		`Vida tras 2 golpes: 70`
		`Dano critico: 11.25`
		`17 split 5 = 3, 17 scrap 5 = 2`
		`Sigue vivo: hit`
		`Sin escudo: miss`
		`Vida igual a 100: hit, distinta de 50: hit`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

2. Entrada de datos y condicionales. Leer datos con chat.load y decidir con scan, fallback y loadout	`1_validos/02_entrada_y_condicionales`. Entrada: Ana y mago	`Nombre del jugador:`
		`Clase (guerrero, mago o arquero):`
		`Bienvenido, Ana. Poder: 95`
		`Rango: Pro`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

3. Ciclos. Usar farm (for), survive (while) y grind (do-while), con ragequit y respawn	`1_validos/03_ciclos`. Entrada: ninguna	`Monedas (1..6 sin el 3): 18`
		`Quedan 3 enemigos`
		`Quedan 2 enemigos`
		`Quedan 1 enemigos`
		`El grind corre al menos una vez`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

4. Funciones, recursión y squad. Combos con y sin retorno, parámetros, recursión directa y operaciones de squad	`1_validos/04_funciones_y_listas`. Entrada: ninguna	`5! = 120`
		`- Tanque`
		`- Sanador`
		`- Arquero`
		`Ahora son 2, el primero es Curandero`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

5. Registros, clases, herencia y manejo de errores. Usar inventory, guild con evolves y clutch/revive/checkpoint	`1_validos/05_objetos_y_errores`. Entrada: ninguna	`Espada tiene poder 40`
		`Conan: POR LA GLORIA!`
		`Error atrapado en la linea 44: division entre cero`
		`El checkpoint siempre corre`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

18.2 Errores sintácticos

Casos de la carpeta `pruebas/2_sintacticos/`.

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

1. Bloque sin cerrar. Detectar un glhf al que le falta su gg	`2_sintacticos/01_bloque_sin_gg`. Entrada: ninguna	`principal.ggez:5:30: error sintactico: bloque sin cerrar: falta 'gg'`
		`posible causa: Cada glhf necesita su gg. Revise que no se haya borrado un gg al final de un bloque.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

2. Combo sin drops. Detectar un combo que no declara su tipo de retorno con drops	`2_sintacticos/02_combo_sin_drops`. Entrada: ninguna	`principal.ggez:5:25: error sintactico: se esperaba 'drops' y se encontro 'glhf'`
		`posible causa: Revise el orden de las palabras en esta linea comparandolo con la sintaxis en Ayuda.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

3. Punto de entrada mal escrito. Detectar que start THE GAME es texto fijo obligatorio	`2_sintacticos/03_punto_de_entrada_mal_escrito`. Entrada: ninguna	`principal.ggez:5:20: error sintactico: se esperaba 'THE' y se encontro 'GAME'`
		`posible causa: Revise el orden de las palabras en esta linea comparandolo con la sintaxis en Ayuda.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

4. Condicional sin glhf. Detectar un scan cuya condición no va seguida de glhf	`2_sintacticos/04_scan_sin_glhf`. Entrada: ninguna	`principal.ggez:7:21: error sintactico: se esperaba 'glhf' y se encontro un salto de linea`
		`posible causa: Revise el orden de las palabras en esta linea comparandolo con la sintaxis en Ayuda.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

5. Declaración sin valor. Detectar un set al que le falta la expresión	`2_sintacticos/05_falta_valor`. Entrada: ninguna	`principal.ggez:6:25: error sintactico: se esperaba una expresion y se encontro un salto de linea`
		`posible causa: Revise el orden de las palabras en esta linea comparandolo con la sintaxis en Ayuda.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

18.3 Errores semánticos

Casos de la carpeta `pruebas/3_semanticos/`.

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

1. Variable no declarada. Detectar el uso de una variable que nunca se declaró con stash	`3_semanticos/01_no_declarada`. Entrada: ninguna	`principal.ggez:7:25: error semantico: 'escudo' no esta declarada`
		`posible causa: Declarela con stash antes de usarla, o revise si escribio mal el nombre o si quedo en otro bloque.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

2. Tipos incompatibles. Detectar que se guarda un lore en una variable score	`3_semanticos/02_tipos_incompatibles`. Entrada: ninguna	`principal.ggez:6:5: error semantico: se esperaba score y se obtuvo lore`
		`posible causa: Los tipos no coinciden: use un valor del tipo declarado (score, aim, lore, win, pixel...).`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

3. Cantidad de argumentos. Detectar una llamada con menos argumentos que parámetros	`3_semanticos/03_cantidad_argumentos`. Entrada: ninguna	`principal.ggez:10:21: error semantico: el combo 'atacar' espera 2 argumento(s) y recibio 1`
		`posible causa: Revise la firma del combo: la cantidad de valores entre { } debe ser igual a la de parametros.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

4. Redeclaración. Detectar dos stash con el mismo nombre en el mismo bloque	`3_semanticos/04_redeclaracion`. Entrada: ninguna	`principal.ggez:7:5: error semantico: 'oro' ya fue declarada en este bloque`
		`posible causa: Para cambiar su valor use set sin stash.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

5. Archivo principal sin punto de entrada. Detectar que el archivo principal no tiene lobby.launch	`3_semanticos/05_sin_punto_de_entrada`. Entrada: ninguna	`principal.ggez:1:1: error semantico: el programa no tiene punto de entrada: falta lobby.launch{start THE GAME}`
		`posible causa: Agregue lobby.launch{start THE GAME} glhf ... gg en el archivo principal.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

18.4 Pruebas de importaciones

Casos de la carpeta `pruebas/4_importaciones/`.

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

1. Importación válida. Usar combos y tipos de un archivo de la misma carpeta (base) y de una subcarpeta (lib)	`4_importaciones/01_importacion_valida`. Entrada: ninguna	`Dano: 60`
		`Pocion cuesta 25 monedas`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

2. Archivo importado inexistente. Detectar un travel a un archivo que no existe	`4_importaciones/02_archivo_inexistente`. Entrada: ninguna	`principal.ggez:5:1: error de importacion: no existe inventario.ggez en la misma carpeta`
		`posible causa: Revise el nombre del archivo o tipo y la carpeta indicada en el from.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

3. Combo inexistente en el módulo. Detectar la llamada a un combo que el archivo importado no define	`4_importaciones/03_combo_inexistente`. Entrada: ninguna	`principal.ggez:8:22: error de importacion: el modulo combate no tiene 'curar'. Tiene: danoTotal`
		`posible causa: Revise el nombre: el mensaje lista lo que si existe.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

4. Tipos incompatibles en una llamada importada. Detectar que se pasa un lore a un parámetro score de un combo de otro archivo	`4_importaciones/04_tipos_en_llamada_importada`. Entrada: ninguna	`principal.ggez:8:33: error de importacion: el parametro 'base' de 'danoTotal' es score y se paso un lore`
		`posible causa: Pase un valor del tipo que declara el parametro.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

5. Error dentro de un archivo importado. Detectar y ubicar un error que está en el archivo importado, no en el principal	`4_importaciones/05_error_en_importado`. Entrada: ninguna	`combate.ggez:2:22: error semantico: 'golpez' no esta declarada`
		`posible causa: Declarela con stash antes de usarla, o revise si escribio mal el nombre o si quedo en otro bloque.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

18.5 Prueba de importación circular

Casos de la carpeta `pruebas/5_circular/`.

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

1. Importación circular. Detectar el ciclo mapa -> zona -> mapa	`5_circular/01_circular`. Entrada: ninguna	`zona.ggez:1:1: error de importacion: importacion en circulo: mapa.ggez -> zona.ggez -> mapa.ggez`
		`posible causa: Elimine uno de los travel del ciclo para que ningun archivo dependa de si mismo.`
		`[codigo de salida 1]`
			Igual al esperado. **Aprobada**

18.6 Prueba completa relacionada con el problema

Casos de la carpeta `pruebas/6_completa/`.

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

1. La aventura del novato. Programa completo de un juego por turnos, como el que haría un estudiante de primer ingreso: módulos, clases con herencia, squad, ciclos, condicionales, recursión, entrada de datos y manejo de errores	`6_completa/01_aventura_del_novato`. Entrada: Ana y mago	`Nombre del heroe:`
		`Clase (guerrero o mago):`
		`== Ana el mago (vida 80, ataque 30) ==`
		`Aparece Slime!`
		`Ana golpea a Slime por 30`
		`Slime derrotado en 1 ronda(s)`
		`Pocion: vida = 110`
		`Aparece Goblin!`
		`Ana golpea a Goblin por 30`
		`Goblin responde por 7`
		`Ana golpea a Goblin por 30`
		`Goblin derrotado en 2 ronda(s)`
		`Pocion: vida = 133`
		`Aparece Dragon!`
		`Ana golpea a Dragon por 30`
		`Dragon responde por 12`
		`Ana golpea a Dragon por 30`
		`Dragon responde por 12`
		`Ana golpea a Dragon por 30`
		`Dragon derrotado en 3 ronda(s)`
		`No se pudo curar: no quedan pociones`
		`Enemigos derrotados: 3 de 3`
		`Experiencia total: 60`
		`Rango final: Pro`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

18.7 Advertencias

Casos de la carpeta `pruebas/7_advertencias/`.

Prueba y objetivo	Entrada	Resultado esperado	Resultado obtenido y estado

1. Advertencias del compilador. Avisar de una variable sin usar, un combo que nunca se llama y código que nunca se ejecuta, sin impedir la ejecución	`7_advertencias/01_advertencias`. Entrada: ninguna	`Oro doble: 100`
		`principal.ggez:11:5: advertencia: este codigo nunca se ejecuta: esta despues de un 'reward'`
		`principal.ggez:16:1: advertencia: la variable 'extra' se declaro pero su valor nunca se usa`
		`principal.ggez:5:1: advertencia: el combo 'sinUso' nunca se llama`
		`[codigo de salida 0]`
			Igual al esperado. **Aprobada**

La prueba completa (`6_completa/01_aventura_del_novato`) es un pequeño juego de rol por turnos, el tipo de programa que un estudiante de primer ingreso querría escribir. Está dividida en tres archivos: `lib/personajes.ggez` con un `inventory` y los `guild` de los personajes (con herencia y

polimorfismo); `reglas.ggez` con los `combo` de las reglas de daño (con recursión y manejo de errores); y `principal.ggez`, que pide el nombre y la clase del jugador con `chat.load`, arma el equipo en un `squad` y simula el combate con ciclos y condicionales.

Además de estas pruebas automáticas, el entorno de desarrollo se verificó manualmente: crear y abrir un proyecto, compilar con `F6`, ejecutar con `F5` con entrada interactiva, ver un error en Problemas con su posible causa y saltar a la línea con un clic, ver advertencias y obtener logros y avanzar misiones. Las capturas de este documento se tomaron durante esas verificaciones.

19. Contribuciones de los integrantes

Esta sección documenta la contribución de cada integrante al desarrollo del proyecto, según el historial del repositorio en GitHub (`https://github.com/VstevenMata/Paradigmas_lenguaje_GGEZ`). Se organiza por integrante e indica los componentes desarrollados y las pruebas realizadas.

Integrante	**Componentes desarrollados**	**Pruebas realizadas**
Steven Mata Quirós	Entorno de desarrollo: ventana principal, menús, explorador de archivos, editor con pestañas y paneles divididos, coloreado, autocompletado, minimapa, búsqueda, terminales integradas, pestañas de errores y problemas, extensiones y temas, terminal de Claude, persistencia de la sesión y ayuda del lenguaje. Modo de juego: maná, niveles, estrellas, monedas, pistas, logros y mundo con misiones. Botones de compilar y ejecutar, archivo principal del proyecto, posible causa de los errores, advertencias, ejecutable único con `publicar.sh` y documentación.	Batería de pruebas de la sección 18 (`pruebas/` y `correr.py`) y verificación manual del entorno de desarrollo.
Maikel Segura Quirós	Primera versión de la terminal integrada. En el traductor: clases, herencia y manejo de errores (`clutch`/`revive`), e importaciones con `travel` (`cargador.py`). Extensión de colores e íconos.	Programas de ejemplo de clases, errores e importaciones.
Angel Ureña Naranjo	Gramática formal (`GRAMATICA.md`). Analizador léxico, sintáctico y semántico, e intérprete: control de flujo, funciones, vectores y registros. Conexión del botón de ejecutar del IDE con el traductor.	Programas de ejemplo del lenguaje (`Traductor/ejemplos/`).

El diseño del lenguaje (vocabulario, tipos y reglas) se discutió y decidió en equipo; las decisiones quedaron registradas en `Traductor/GRAMATICA.md`.

20. Conclusiones

Esta sección presenta las conclusiones del proyecto, respondiendo a los objetivos planteados en la sección 5 y reflexionando sobre las decisiones de diseño, lo que se ajustaría en una segunda iteración y las limitaciones encontradas.

Los objetivos de la sección 5.2 se cumplieron. Definimos formalmente el lenguaje, con 71 palabras reservadas tomadas de los videojuegos, cinco tipos simples, dos compuestos, clases y reglas precisas de alcance y de importación. El compilador realiza las cuatro fases descritas en la sección 13, incluida la detección de importaciones circulares, y el intérprete ejecuta el árbol sintáctico sin recurrir a `eval` ni `exec`. El entorno de desarrollo propio integra editor, árbol de archivos, terminal, pestañas de errores y problemas, consulta del lenguaje y botones de compilar y ejecutar, y la gamificación ligada a la ejecución cumple el papel de innovación. Como evidencia, las 23 pruebas de la sección 18 se aprueban y el aplicativo se genera como un ejecutable único.

Entre las decisiones de diseño, consideramos acertadas las que acercan el lenguaje al estudiante sin alejarlo de los lenguajes profesionales: el vocabulario de videojuegos sobre una estructura igual a la de Java, los bloques `glhf`/`gg` en lugar de la indentación significativa, los operadores escritos con palabras (que eliminan la confusión entre = y ==), el tipado estático que detecta errores antes de ejecutar, y los mensajes de error con posible causa. Separar el traductor del IDE y comunicarlos con un formato de texto simple permitió trabajar en paralelo y probar el traductor de forma automática.

En una segunda iteración, acordaríamos desde el inicio el formato de los errores y la batería de pruebas, que en este proyecto se completaron al final, y probaríamos antes el ejecutable en Windows. También implementaríamos lo que quedó reservado (interfaces, pilas, colas y la biblioteca `arsenal`), la conversión de texto a número, el formateador automático y un depurador paso a paso.

Reconocemos varias limitaciones. GGEZ no se ha evaluado con estudiantes de primer ingreso, por lo que su efecto sobre la motivación se basa en la observación del equipo y no en una medición. El ejecutable de Windows no se ha probado en un equipo real, el intérprete es lento para programas grandes y la recursión se limita a 5000 llamadas anidadas.

21. Referencias

El proyecto no utilizó fuentes bibliográficas externas. A continuación se listan los documentos del curso y del equipo que guiaron el trabajo, y las herramientas de software utilizadas.

Documentos

Universidad Nacional. (2026). *Descripción del Proyecto 2026* [Enunciado del proyecto del curso Paradigmas de Programación]. Sede Regional Brunca.

Problemas de traducción de lenguajes. (s.f.). [Material del curso Paradigmas de Programación, Universidad Nacional].

Mata Quirós, S., Segura Quirós, M. y Ureña Naranjo, A. (2026). *Propuesta de Lenguaje GGEZ* [Documento interno del equipo].

Mata Quirós, S., Segura Quirós, M. y Ureña Naranjo, A. (2026). *Gramática formal de GGEZ* [Archivo `Traductor/GRAMATICA.md` del repositorio del proyecto].

Herramientas de software

AvaloniaUI. (s.f.). *Avalonia* (versión 12.1.1) [Software]. `https://avaloniaui.net`

Microsoft. (s.f.). *.NET* [Software]. `https://dotnet.microsoft.com`

Microsoft. (s.f.). *CommunityToolkit.Mvvm* (versión 8.4.2) [Software]. `https://github.com/CommunityToolkit/dotnet`

RoyalApps. (s.f.). *RoyalApps.RoyalTerminal.Avalonia* (versión 0.5.2) [Software]. `https://www.nuget.org/packages/RoyalApps.RoyalTerminal.Avalonia`

Python Software Foundation. (2024). *Python* (versión 3.12) [Software]. `https://www.python.org`

Astral. (2024). *python-build-standalone* [Software]. `https://github.com/astral-sh/python-build-standalone`

22. Anexos

22.1 Manual de usuario de GGEZ

Instalación

GGEZ se entrega como un único archivo ejecutable, generado con `publicar.sh`. No requiere instalar .NET ni Python.

- **Linux**: dar permiso de ejecución y abrirlo: `chmod +x GGEZ` y luego `./GGEZ`.
- **Windows**: abrir `GGEZ.exe`.

La primera vez, el IDE despliega el traductor y su Python en la carpeta de datos del usuario; esto toma unos segundos y no se repite. Para generar el ejecutable desde el código fuente: `./publicar.sh linux`, `./publicar.sh windows` o `./publicar.sh todo`; el resultado queda en la carpeta `dist/`.

La ventana principal

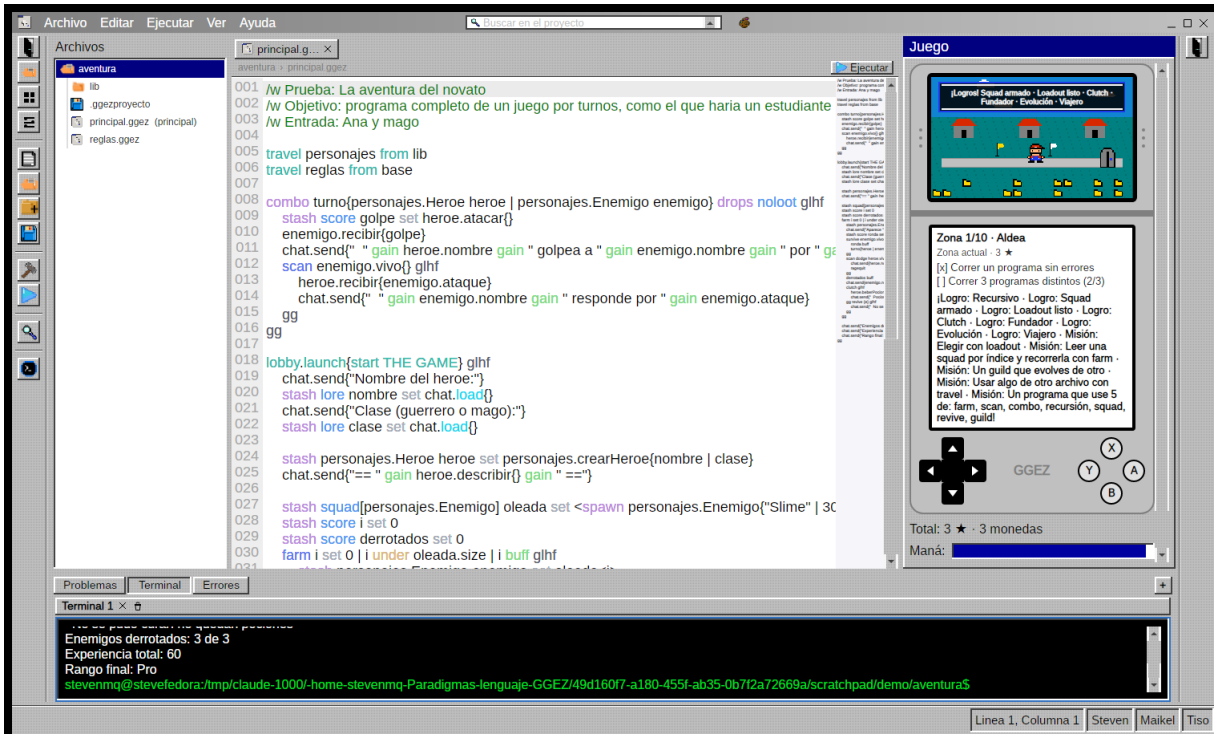


Figura 6: Ventana principal del IDE.

- **Barra de menú** (arriba): Archivo, Editar, Ejecutar, Ver y Ayuda.
- **Barra lateral** (izquierda): íconos para cambiar entre el explorador de archivos, las extensiones y los ajustes.
- **Explorador**: el árbol de archivos del proyecto abierto.

- **Editor** (centro): pestañas con los archivos abiertos, números de línea, coloreado de sintaxis y minimapa a la derecha. Sobre el editor están la ruta del archivo y los botones de compilar (martillo) y ejecutar (reproducir).
- **Segmento de salida** (abajo): pestañas Terminal, Errores y Problemas.
- **Panel de juego** (derecha): maná, mapa del mundo, nivel actual, monedas, pistas y logros.
- **Barra de estado** (abajo del todo): línea y columna del cursor y otros datos.

Crear y abrir un proyecto

1. Menú **Archivo** → **Abrir carpeta** (`Ctrl+O`) y elegir o crear una carpeta.
2. En el explorador, clic derecho para crear archivos (`.ggez`) y carpetas, renombrar o eliminar.
3. Opcional: clic derecho sobre un archivo con `lobby.launch` → **Marcar como principal**. El archivo aparece con la etiqueta «(principal)» y se usa cuando la pestaña enfocada no es un `.ggez`.

Sin proyecto abierto, también se puede crear un archivo suelto con `Ctrl+N`, guardarlo y ejecutarlo.

Escribir código

- El editor colorea palabras reservadas, tipos, operadores, textos y comentarios.
- **Autocompletado**: aparece al escribir o con `Ctrl+Espacio`. Sugiere palabras reservadas, plantillas completas (por ejemplo, un `farm` con sus tres partes) con su descripción, los nombres declarados en el archivo y, después de un punto, los miembros disponibles (`chat.send`, `lista.recruit`, lo de un módulo importado). Con las flechas se elige y con `Enter` o `Tab` se inserta; `Esc` cierra la lista.
- **Minimapa**: muestra todo el archivo en miniatura a la derecha del editor; un clic salta a esa parte.
- Con la extensión de colores de bloques, al presionar `Enter` después de `glhf` el editor cierra el bloque con `gg`.
- `Ctrl+F` busca, `Ctrl+H` reemplaza y `Ctrl+G` va a una línea.

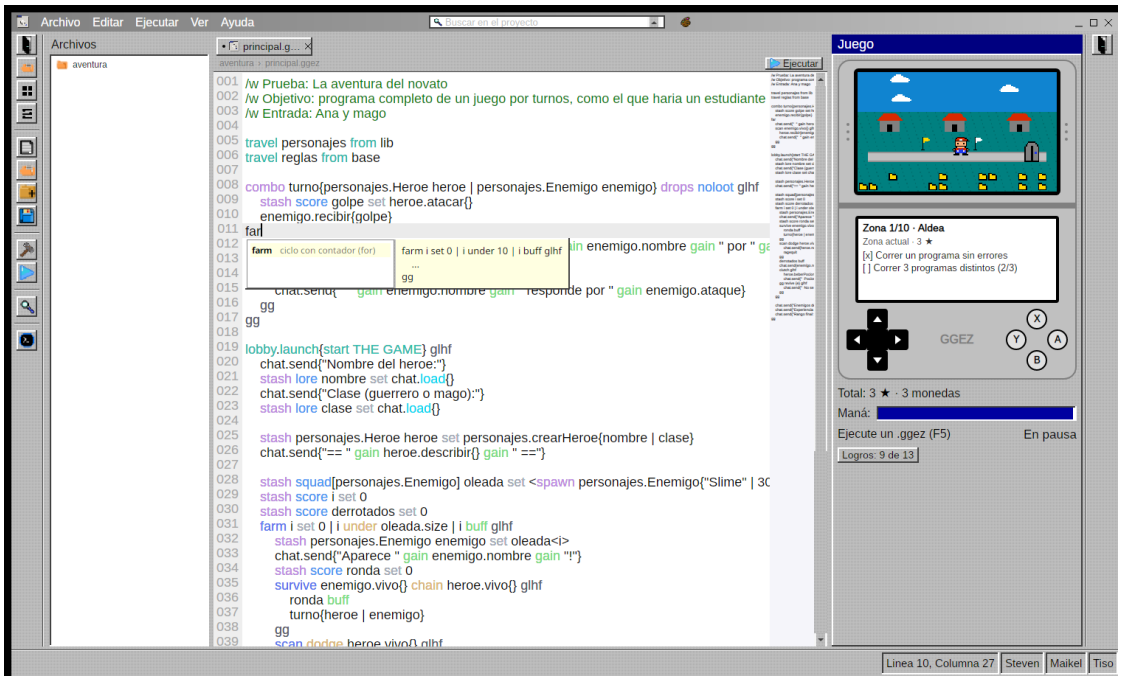


Figura 7: Autocompletado con plantillas y su descripción.

Compilar y ejecutar

- **Compilar** (`F6` o el martillo): revisa el archivo `.ggez` de la pestaña enfocada sin ejecutarlo. Si todo está bien, la terminal muestra «Compilacion exitosa».
- **Ejecutar** (`F5` o el botón de reproducir): compila y, si no hay errores, ejecuta el archivo `.ggez` de la pestaña enfocada. Cuando el programa pide datos con `chat.load`, se escriben en la terminal y se presiona `Enter`.
- `Ctrl+C` en la terminal detiene un programa en ejecución.

Corregir errores

1. Si hay un error, la terminal lo muestra en rojo y la pestaña **Problemas** lo lista con su tipo, archivo, línea y posible causa.
2. Un clic sobre el problema abre el archivo y lleva el cursor a la línea.
3. Las advertencias aparecen en amarillo y no impiden ejecutar.
4. Menú **Ejecutar** → **Ver errores** abre la pestaña de errores.

Terminal

La terminal permite escribir comandos del sistema y tiene comandos propios: `cd`, `clear`, `exit` y `help`. Con el botón `+` se abren hasta cuatro terminales, una al lado de la otra. Con la extensión de Claude instalada, las terminales son reales y permiten ejecutar el asistente `claude`.

Modo de juego

- **Maná**: la barra muestra cuánto gastó el último programa.
- **Mapa**: muestra la zona actual y sus misiones; las flechas recorren las zonas. Al cumplir todas las misiones de una zona se abre la siguiente.
- **Niveles**: abrir la carpeta de un nivel (con `nivel.json`) y ejecutar el archivo; el panel muestra cuántas pruebas pasaron y las estrellas ganadas.
- **Pistas**: el botón de pista gasta una moneda y muestra la siguiente pista del nivel.
- **Logros**: la lista muestra los obtenidos y los pendientes.

Extensiones y ajustes

En la vista de **Extensiones** se instalan y desinstalan complementos con un clic, entre ellos los temas desbloqueados con estrellas. En **Ajustes** se cambian el tamaño de letra, el zoom y otras preferencias del editor. Todo se conserva al cerrar el IDE, junto con las pestañas abiertas.

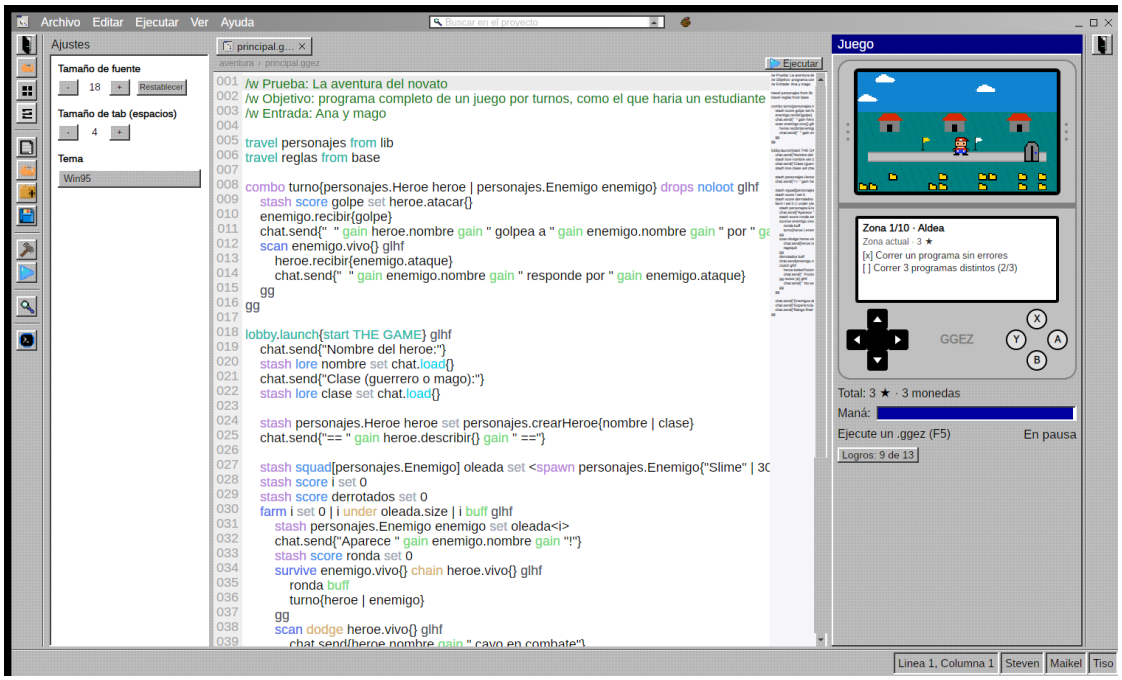


Figura 8: Vista de ajustes.

Ayuda

- **Ayuda → Sintaxis de GGEZ**: referencia completa del lenguaje, con cada ejemplo junto a su equivalente en Java.
- **Ayuda → Atajos de teclado** (`F1`): lista de todos los atajos.
- **Ayuda → Acerca de**: versión e integrantes.

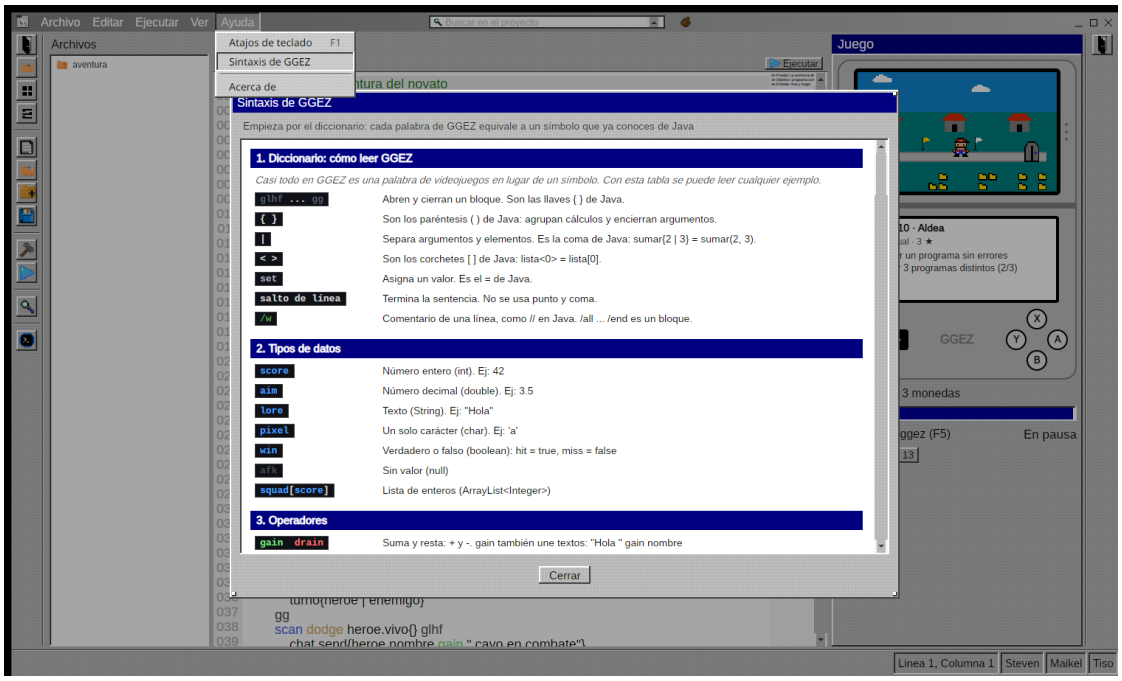


Figura 9: Referencia del lenguaje con el equivalente en Java.

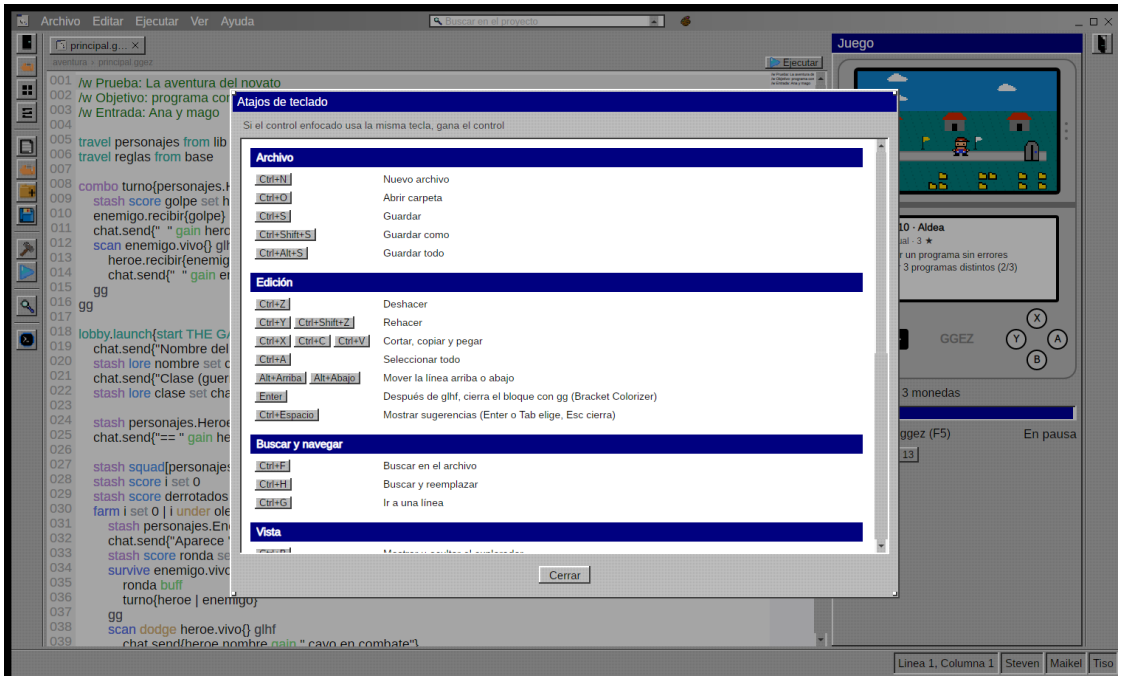


Figura 10: Atajos de teclado.

Atajos principales

Atajo	Acción
F5 / F6	Ejecutar / Compilar
F1	Mostrar los atajos

Atajo	Acción
`Ctrl+N`, `Ctrl+O`	Nuevo archivo, abrir carpeta
`Ctrl+S`, `Ctrl+Alt+S`	Guardar, guardar todo
`Ctrl+Z`, `Ctrl+Y`	Deshacer, rehacer
`Alt+Arriba` / `Alt+Abajo`	Mover la línea
`Ctrl+Espacio`	Sugerencias de autocompletado
`Ctrl+F`, `Ctrl+H`, `Ctrl+G`	Buscar, reemplazar, ir a línea
`Ctrl+B`, `Ctrl+J`	Mostrar u ocultar explorador y terminal
`Ctrl++`, `Ctrl+-`, `Ctrl+0`	Zoom
`Ctrl+C` (en la terminal)	Interrumpir el programa